



Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Parky - Marketplace colaborativo de aparcamiento Peer-to-Peer

Peer-to-Peer Collaborative Parking Marketplace

Hugo González Pérez

La Laguna, 14 de junio de 2026

D. **Francisco Javier Rodríguez González**, con N.I.F. 43.618.712-V profesor Asociado de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor

D. **Alejandro Pérez Nava**, con N.I.F. 43.821.179-S profesor Asociado de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como cotutor

C E R T I F I C A (N)

Que la presente memoria titulada:

“Parky - Marketplace colaborativo de aparcamiento Peer-to-Peer”

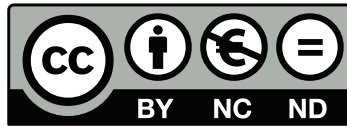
ha sido realizada bajo su dirección por D. **Hugo González Pérez**, con N.I.F 78.824.986-F.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 14 de junio de 2026

Agradecimientos

A mi familia, especialmente a mis padres, por su apoyo incondicional y por creer siempre en mí, dándome la fuerza necesaria para superar cada etapa de mi formación académica. Sin su sacrificio y cariño, nada de esto habría sido posible. A mis amigos y compañeros, por los momentos compartidos, las largas jornadas de estudio y por hacer que este camino fuera mucho más ameno. Por último, a la Universidad de La Laguna por brindarme el entorno y las herramientas necesarias para formarme como profesional.

Licencia



© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional.

Resumen

En la última década, el incremento del número de vehículos y, sobre todo, del turismo en muchas ciudades canarias ha supuesto una mayor dificultad tanto para el tránsito como para encontrar aparcamiento, sobre todo en las zonas urbanas y en las zonas turísticas. Esto trae problemas tales como la congestión del tráfico, tiempo perdido para los conductores y uso ineficiente del espacio disponible. Al mismo tiempo, numerosas plazas de aparcamiento privadas permanecen vacías durante largos periodos de tiempo.

En este sentido, las plataformas digitales que se sustentan en la economía colaborativa se han transformado en una herramienta útil para conectar personas que tienen un recurso que ofrecer con aquellas que lo necesitan (p. ej., Airbnb, BlaBlaCar, Wallapop).

Por ello, en el presente Trabajo de Fin de Grado se propone el desarrollo de un prototipo de aplicación Full Stack para el alquiler de plazas de aparcamiento privadas. La plataforma operará como un marketplace que conectará a propietarios de plazas (casas particulares o comunidades) con conductores que requieran estacionar su vehículo, facilitando la gestión de reservas y la disponibilidad de los aparcamientos.

Desde el aspecto técnico, el sistema se ha desarrollado con React 18 como framework frontend, TypeScript en modo estricto y Tailwind CSS para el diseño de interfaz, empleando Vite como herramienta de compilación y Supabase como backend gestionado, que proporciona base de datos PostgreSQL, autenticación con OAuth social y almacenamiento de ficheros. La seguridad de los datos se garantiza mediante políticas de Row Level Security (RLS) y triggers PL/pgSQL que aseguran la integridad de las reservas a nivel de base de datos. La búsqueda y localización de plazas se implementan mediante Leaflet y el servicio de geocodificación OpenCage. El sistema incluye un módulo de acceso físico digital (SmartAccess) vinculado al estado de cada reserva, y una arquitectura de pagos P2P mediante Stripe Connect que contempla la gestión del IGIC canario. La aplicación se despliega simultáneamente como Single Page Application en Vercel y como aplicación Android nativa mediante Capacitor.

Palabras clave: Marketplace colaborativo, P2P, aparcamiento urbano, React, TypeScript, Supabase, PostgreSQL, Stripe Connect, Capacitor, Row Level Security.

Abstract

Over the past decade, the increase in the number of vehicles and, above all, in tourism in many Canary Islands cities has made traffic and finding parking more difficult, especially in urban and tourist areas. This leads to problems such as traffic congestion, wasted time for drivers, and inefficient use of available space. At the same time, numerous private parking spaces remain empty for long periods of time.

In this regard, digital platforms based on the sharing economy have become a useful tool for connecting people who have a resource to offer with those who need it (e.g., Airbnb, BlaBlaCar, Wallapop).

Therefore, this Final Degree Project proposes the development of a full-stack application prototype for the rental of private parking spaces. The platform will operate as a marketplace connecting parking space owners (private homes or communities) with drivers who need to park their vehicles, facilitating the management of reservations and the availability of parking spaces.

From a technical standpoint, the system has been developed using React 18 as the frontend framework, TypeScript in strict mode, and Tailwind CSS for interface design, employing Vite as the build tool, which provides a PostgreSQL database, social OAuth authentication, and file storage. Data security is ensured through Row Level Security (RLS) policies and PL/pgSQL triggers that guarantee the integrity of reservations at the database level. Searching for and locating parking spaces is implemented using Leaflet and the OpenCage geocoding service. The system includes a digital physical access module (SmartAccess) linked to the status of each reservation, and a P2P payment architecture using Stripe Connect that handles the Canary Islands IGIC tax. The application is deployed simultaneously as a Single Page Application on Vercel and as a native Android app using Capacitor.

Keywords: Collaborative marketplace, P2P, urban parking, React, TypeScript, Supabase, PostgreSQL, Stripe Connect, Capacitor, Row Level Security.

Índice general

Capítulo 1 Introducción	1
1.1 Definición del problema	1
1.2 Justificación	2
1.3 Estado del arte	2
1.3.1 Aplicaciones de zona regulada	2
1.3.2 Plataformas de reserva en aparcamientos comerciales	3
1.3.3 Plataformas P2P de referencia internacional	3
1.3.4 Iniciativas P2P en el mercado nacional	4
1.3.5 Análisis comparativo	4
1.4 Tendencias del mercado	5
1.4.1 El mercado de aparcamiento en España	5
1.4.2 El mercado en Canarias y el área metropolitana de Tenerife	7
1.4.3 La Zona de Bajas Emisiones de Santa Cruz de Tenerife	8
1.4.4 Disponibilidad y precios en el modelo P2P	8
1.5 Objetivos	8
Capítulo 2 Estudio de tecnologías y decisiones de diseño	10
2.1 Lenguajes de programación y frameworks	10
2.1.1 Frontend	10
2.1.2 APIs externas de cartografía y geocodificación	12
2.1.3 Backend	12
2.1.4 Base de datos	13
2.2 Entorno de desarrollo	13
2.3 Plataformas de despliegue	14
2.4 Análisis de alternativas y decisiones de diseño	14
Capítulo 3 Metodología y análisis de requisitos	16
3.1 Metodología de desarrollo	16
3.2 Actores del sistema	17
3.3 Requisitos funcionales	18
3.4 Requisitos no funcionales	18
3.5 Modelo de casos de uso	20
3.6 Trazabilidad de objetivos	23
3.7 Diseño de la base de datos	23
3.7.1 Diagrama de la base de datos	24
3.7.2 Políticas RLS	24
3.7.3 Trigger de integridad de reservas	25
Capítulo 4 Estudio de viabilidad económica	26
4.1 Del prototipo al producto comercial	26
4.2 Funcionalidades de la versión avanzada	27
4.3 Planificación y coste de la versión comercial	28
4.3.1 Equipo y coste del proyecto	30

4.4	Modelo de comercialización	32
4.4.1	Comisión transaccional asimétrica	32
4.4.2	Parky Pro: suscripción para arrendadores con cartera	32
4.4.3	Visibilidad premium	33
4.5	Estimación del retorno de la inversión	33
4.5.1	Hipótesis de partida	33
4.5.2	Proyección y punto de equilibrio	35
4.5.3	Análisis de sensibilidad	37
Capítulo 5	Implementación	39
5.1	Arquitectura por capas	39
5.2	Módulo de autenticación	39
5.3	Módulo de plazas y mapa	39
5.4	Módulo de reservas	40
5.5	Módulo de pagos	41
5.6	Módulo SmartAccess	41
5.6.1	Modelo de acceso físico	41
5.6.2	Gestión de incidencias y casos límite	42
5.7	Despliegue multiplataforma	43
5.8	Pruebas realizadas	43
Capítulo 6	Conclusiones y líneas futuras	45
6.1	Líneas de trabajo futuro	45
Capítulo 7	Summary and Conclusions	46
7.1	Future work	47
Capítulo 8	Presupuesto	48
8.1	Recursos humanos	48
8.2	Equipos y material	49
8.3	Herramientas de software	49
8.4	Coste total	49

Índice de figuras

1.1	Evolución del parque de vehículos en España 1990–2025. Fuente: DGT [19].	6
1.2	Evolución anual del parque de vehículos en Canarias (datos desde 1961). Fuente: ISTAC [23].	7
2.1	Comparativa entre el prototipo en Figma (izquierda) y la pantalla principal de Parky en producción (derecha).	14
3.1	Diagrama de casos de uso del sistema Parky.	22
3.2	Diagrama entidad-relación del esquema de base de datos de Parky.	24
4.1	Coste del proyecto Parky exportado desde ProjectLibre (enero 2026 – marzo 2027) .	29
4.2	Diagrama de Gantt del proyecto Parky exportado desde ProjectLibre (enero 2026 – marzo 2027)	30
4.3	Evolución del inventario de plazas activas y la demanda por plaza a lo largo de los veinticuatro meses proyectados.	34
4.4	Desglose mensual de ingresos por vía de monetización. La línea discontinua roja representa los gastos operativos mensuales.	36
4.5	Evolución del balance acumulado a lo largo de los veinticuatro meses.	37

Índice de tablas

1.1	Comparativa de plataformas de aparcamiento	5
1.2	Comparativa de tarifas de aparcamiento regulado por ciudad y zona.	6
2.1	Comparativa de frameworks JavaScript de frontend para el desarrollo de Parky. . . .	11
2.2	Comparativa de plataformas BaaS frente a las necesidades de Parky.	15
3.1	Sprints de desarrollo de Parky (enero–mayo de 2026).	17
3.2	Actores del sistema Parky.	18
3.3	Requisitos funcionales de Parky.	19
3.4	Requisitos no funcionales de Parky.	20
3.5	CU-01: Autenticarse y obtener rol.	20
3.6	CU-02: Publicar plaza de garaje.	21
3.7	CU-03: Buscar y filtrar plazas en el mapa.	21
3.8	CU-04: Realizar reserva y pago.	22
3.9	Matriz de trazabilidad: objetivo específico → req. funcional → caso de uso.	23
3.10	Políticas de seguridad a nivel de fila (RLS) del esquema de Parky.	25
4.1	Presupuesto por perfil técnico.	30
4.2	Recursos materiales y coste total del proyecto.	31
4.3	Desglose de la inversión inicial recomendada para Parky.	31
4.4	Parámetros clave del modelo de proyección financiera de Parky.	35
4.5	Evolución proyectada de ingresos y gastos acumulados. La fila sombreada en gris (mes 3) es el primer mes con flujo neto positivo; la fila en verde (mes 19) marca la recuperación íntegra de la inversión inicial de 287.000,00 €.	36
4.6	Análisis de sensibilidad del ROI bajo cuatro escenarios de operación.	37
8.1	Desglose de horas y coste por fase.	48
8.2	Resumen del coste total del proyecto.	49

Capítulo 1

Introducción

1.1. Definición del problema

No es una novedad que el crecimiento del *parque móvil*¹ y el auge del turismo en las Islas Canarias y sobre todo en Tenerife han provocado que encontrar aparcamiento en prácticamente cualquier lugar resulte cada vez más difícil. En las zonas de alta densidad, como puede ser el **área metropolitana**, el problema es aún más acentuado, donde tanto residentes como visitantes diariamente sufren un desequilibrio crítico entre oferta y demanda de espacio urbano.

En muchas vías de Tenerife, este problema se manifiesta de forma recurrente en los principales ejes urbanos. Cualquier conductor que transite por la zona que hay entre Santa Cruz de Tenerife y San Cristóbal de La Laguna puede verse atrapado en atascos que, en trayectos de apenas diez kilómetros, generan pérdidas de tiempo significativas y degradan la calidad de vida. Sin embargo, el problema no reside únicamente en la congestión de vías principales como la *(TF-5)*², sino en la dificultad añadida de encontrar un hueco libre al llegar al destino. De hecho, se estima que hasta un 35 % del tráfico urbano en horas punta está compuesto por vehículos que circulan en bucle buscando estacionamiento [1], lo que contribuye innecesariamente a la contaminación acústica y a las emisiones de gases de efecto invernadero (*GEI*)³.

Moverse por Santa Cruz de Tenerife en hora punta se ha convertido en una prueba de paciencia: el *TomTom Traffic Index* [2] registró en 2025 que los conductores perdieron una media de 52 horas atrapados en el tráfico y un índice de congestión del 34,8 %, lo que equivale a pasar más de dos días enteros al año prácticamente parados en cola. El gran embudo diario llega con el atardecer, cuando la velocidad cae a unos desesperantes 28 km/h y la congestión alcanza el 55,7 %.

Resulta irónico que, mientras cientos de conductores pierden tiempo y combustible buscando aparcamiento, estas plazas permanezcan vacías durante gran parte del día o incluso durante periodos vacacionales. Este desaprovechamiento de un recurso escaso pone de manifiesto una clara ineficiencia en la gestión del espacio disponible.

Es por ello que, a partir de este gran contraste, nace la motivación de **Parky** y su principal objetivo: explorar cómo se puede conectar a los propietarios de estas plazas privadas poco usadas con conductores que requieren estacionar su vehículo, facilitando por medio de tecnologías digitales una gestión más inteligente y eficiente de los recursos urbanos existentes.

Nota terminológica: a lo largo de la memoria, los términos *arrendador* y *propietario de plaza* se emplean de forma equivalente para referirse al titular del garaje que lo pone en alquiler; del mismo modo, *arrendatario* y *conductor* designan indistintamente a quien realiza la reserva.

¹**Parque móvil:** Conjunto total de vehículos registrados y en circulación en una determinada región o país.

²**TF-5:** Es la principal autopista del norte de Tenerife, que conecta Santa Cruz de Tenerife con el área metropolitana y los municipios del norte de la isla.

³**GEI:** Los gases de efecto invernadero son aquellos que contribuyen al calentamiento global al retener el calor en la atmósfera, como el dióxido de carbono (CO₂).

1.2. Justificación

Parky busca responder a tres factores que convergen en el contexto urbano de 2026: presión normativa, eficiencia económica y madurez tecnológica.

En primer lugar, desde la perspectiva de la **sostenibilidad y presión regulatoria**, el proyecto responde a la obligatoriedad de implementar Zonas de Bajas Emisiones (*ZBE*) en municipios de más de 50.000 habitantes, como es el caso de Santa Cruz de Tenerife y San Cristóbal de La Laguna. Una parte sustancial del tráfico urbano es generado exclusivamente por la búsqueda de estacionamiento. Facilitar una plataforma (*P2P*)⁴ permite una transición hacia una movilidad más ordenada, reduciendo significativamente las emisiones de gases y el ruido en el centro de las ciudades, alineándose así con los objetivos de las denominadas *Smart Cities*⁵.

En segundo lugar, la **eficiencia social y económica** del modelo P2P está respaldada por datos: la movilidad compartida registra una tasa de crecimiento anual compuesta (*CAGR*)⁶ del 14,8 % hasta 2030 [3]. **Parky** permite a los propietarios obtener entre 150 € y 300 € mensuales por una plaza infrautilizada, mientras que ofrece al conductor un ahorro directo de tiempo ya que se dedica de media 24 horas anuales solo a buscar hueco [4].

Por último, la **viabilidad tecnológica** está respaldada por la madurez del ecosistema disponible. Las APIs de geolocalización, las pasarelas de pago P2P y las bases de datos con seguridad a nivel de fila existen, están documentadas y tienen precios accesibles para un prototipo. El mismo conductor que hoy circula en bucle puede reservar y acceder a una plaza privada en menos de dos minutos desde el móvil.

1.3. Estado del arte

Las plataformas de aparcamiento en 2026 se organizan en cuatro categorías con modelos de negocio y barreras de entrada distintas que se describen a continuación.

1.3.1. Aplicaciones de zona regulada

Las plataformas de zona regulada digitalizan el cobro de tasas de estacionamiento en vía pública mediante contratos con los ayuntamientos. Sin ese convenio, no pueden operar en ninguna ciudad. Esta dependencia institucional define su modelo de expansión y limita su presencia a los municipios que han licitado el servicio.

EasyPark [5] opera en más de 4.200 ciudades de 21 países, con más de 6.000 operadores de aparcamiento integrados, resultado de una estrategia de adquisiciones de otras plataformas de este mercado. La filial española, con sede en Barcelona, gestiona la zona regulada en más de 130 ciudades

⁴**P2P**: Sistema digital que permite a personas o empresas realizar transacciones directas entre sí, sin la necesidad de intermediarios tradicionales como bancos o grandes empresas.

⁵**Smart Cities**: es un entorno urbano que utiliza tecnologías avanzadas, como el Internet de las Cosas (IoT), la inteligencia artificial (IA), el big data y las TIC (Tecnologías de la Información y Comunicación), para mejorar la calidad de vida de sus habitantes, optimizar la gestión de recursos y promover el desarrollo sostenible.

⁶**CAGR** (*Compound Annual Growth Rate*): indicador financiero que mide la tasa de crecimiento anual compuesta de una inversión o mercado durante un período determinado, asumiendo la reinversión de los beneficios.

de la península actualmente [6]. Cabe destacar que su cobertura de zona regulada en el archipiélago es nula ya que no cuenta con ningún convenio con ayuntamientos de las islas para el pago digital de aparcamientos regulados.

Telpark [7] es la marca comercial del grupo Empark, segundo operador de movilidad urbana en la península ibérica con 333.000 plazas distribuidas en 150 ciudades de España, Portugal, Andorra y Turquía. En las Islas Canarias, Telpark gestiona únicamente el aparcamiento subterráneo Ramón y Cajal en Tenerife y en Gran Canaria gestiona 3 parkings públicos de la capital [8].

1.3.2. Plataformas de reserva en aparcamientos comerciales

Este segmento conecta conductores con aparcamientos privados de rotación: parkings de centros comerciales, hoteles y edificios de oficinas. El inventario son instalaciones profesionales con gestión propia, no plazas de particulares.

Pardclick [9] es una plataforma de reservas de aparcamiento que mantiene integración tecnológica con el grupo francés Indigo y su red Parkia. Opera como *marketplace* de reservas B2C⁷, permitiendo al usuario final reservar plazas en instalaciones Indigo/Parkia con acceso automatizado. Cuenta con más de 3000 parkings en más de 500 ciudades europeas. Sin embargo, en Tenerife, su única instalación es el parking La Trinidad cerca del Centro Histórico en San Cristóbal de La Laguna.

EIParking [10] pertenece al Grupo Mutua Madrileña desde 2021. Con más de 3,5 millones de usuarios, es la aplicación multimodal de servicios al conductor más utilizada de España. Sus ingresos combinan la gestión de tickets de zona regulada en varias ciudades, reservas en parkings off-street, intermediación en gasolineras y puntos de carga, el dispositivo *Via-T* de telepeaje usado en península y la gestión de citas *ITV*. En Tenerife ofrece valet en los dos aeropuertos de la isla (TF Norte y TF Sur) a través del operador local Car&Fly, y gestiona reservas en el parking privado La Multa (próximo al Hospital Universitario).

1.3.3. Plataformas P2P de referencia internacional

Las plataformas P2P de mayor escala prueban que el modelo entre particulares funciona. Se han consolidado en mercados anglófonos y **ninguna opera en España a día de hoy**.

JustPark [11] nació en Londres en 2006. En 2024 fue adquirida por *ParkHub* [12], consolidándose bajo marca única en 2025 y en marzo de 2026 adquirió *BigParking*. La plataforma cuenta con 14 millones de conductores y gestiona 250.000 plazas, de las que unas 50.000 son espacios residenciales P2P. La plataforma documenta [13] que sus propietarios en el Reino Unido obtienen entre 500 y 3.000 libras anuales por espacio publicado, según la ubicación y la demanda del entorno.

SpotHero [14] se fundó en Chicago en 2011 y acumula más de 1.000 millones de dólares en transacciones sobre un catálogo de más de 13.000 garajes en más de 400 ciudades norteamericanas. El 23 de febrero de 2026, *Uber* anunció su adquisición [15], de cara a usar las plazas como nodos

⁷**B2C** (Business to Consumer): Modelo de negocio en el que las empresas venden productos o servicios directamente al consumidor final para uso personal, sin intermediarios.

de *staging*⁸ para que su flota autónoma actúen como robotaxis. Para ese propósito, la plataforma desarrolló *HeroConnect*, que permite la comunicación directa entre el ordenador de a bordo del vehículo y las barreras electromecánicas del parking aunque los conductores convencionales acceden mediante códigos de barras o QR digitales.

1.3.4. Iniciativas P2P en el mercado nacional

En España han surgido dos proyectos que identifican el mismo nicho que Parky, con tracción de mercado muy limitada.

ReservPark [16] combina un sistema de intercambio colaborativo de plazas en vía pública (gratuito, basado en avisos) con un módulo de alquiler de garajes privados por horas (desde 2 €/hora) con pago digital mediante Apple Pay y Google Pay. En mayo de 2026 registraba únicamente en torno a 100 descargas en Google Play y 7 valoraciones en App Store, y a día de hoy no tiene inventario ni usuarios activos en Canarias.

SocialPark [17] es una plataforma en fase de pruebas desarrollada por Javier Aparicio León que permite comprar y vender plazas de aparcamiento públicas en tiempo real. Los conductores que van a liberar su plaza en calle pueden ofrecerla a través de la app y recibir una recompensa en *ParkCoins* (la moneda virtual propia de la plataforma) o mediante pequeñas compensaciones económicas directas. En mayo de 2026 contaba con pocas descargas y valoraciones en Google Play y App Store, pero aún no se ha consolidado en Santa Cruz de Tenerife a pesar de estar desarrollada para esta ciudad.

Que el único proyecto P2P con anclaje explícito en Santa Cruz de Tenerife sea esta última plataforma experimental confirma que la demanda existe, pero que no se ha identificado una solución consolidada con oferta organizada y acceso automatizado.

1.3.5. Análisis comparativo

El resultado es el mismo en todos los casos: ninguna plataforma resuelve los tres componentes del problema a la vez. Las profesionales (EasyPark, Telpark, Parclick, ElParking) cubren descubrimiento y pago, pero trabajan con inventario comercial y tienen poca o nula cobertura en Canarias. Las P2P internacionales (JustPark, SpotHero) prueban que el modelo escala entre particulares, pero no operan en España y no contemplan la fiscalidad canaria. Las nacionales (ReservPark, SocialPark) identifican el nicho pero se quedan en el descubrimiento: el acceso físico queda delegado a un chat o a la entrega de llaves. Esto queda reflejado en la Tabla 1.1 la cual resume las variables clave de cada plataforma frente a *Parky*.

La diferencia entre plataformas con retención alta (Telpark con **Entrada Express**⁹, ElParking con botón in-app y Via-T, SpotHero con HeroConnect) y las que pierden usuarios en el último paso (JustPark, ReservPark, SocialPark) es una sola: las primeras eliminan la fricción en el acceso, las segundas la transfieren al usuario. Para un garaje particular, que el propietario esté localizable para abrir la barrera no es una solución operativa ya que impide reservas simultáneas.

⁸**Staging** (en flotas autónomas): área física donde los vehículos autónomos esperan entre servicios, realizan operaciones de mantenimiento automatizado (recarga de baterías, limpieza, diagnóstico) y reciben la asignación de los próximos trayectos.

⁹**Entrada Express**: Sistema sin contacto que permite el acceso a sus parkings mediante el reconocimiento automático de la matrícula, vinculada a la cuenta del usuario

La barrera fiscal del **IGIC**¹⁰ añade otro obstáculo para plataformas peninsulares. Ninguna ha implementado integración con ese régimen fiscal. Para un desarrollador de Madrid, un mercado de 1,2 millones de personas no justifica una integración fiscal propia. Para Parky, esa misma especificidad constituye una ventaja competitiva potencial: quien resuelva el IGIC de forma nativa reduce significativamente la barrera de entrada para competidores externos antes de que estos establezcan presencia en el mercado local.

Parky cubre ese espacio: plataforma P2P diseñada para el área metropolitana de Tenerife que integra descubrimiento, pago y acceso físico en un solo flujo, con gestión del IGIC integrada.

Tabla 1.1: Comparativa de plataformas de aparcamiento

Plataforma	P2P	Mapa	Pago integrado	App nativa	Acceso digital	Canarias
EasyPark	No	Sí	Sí	Sí	No	No
Telpark	No	Sí	Sí	Sí	Sí	Parcial
Parclick	No	Sí	Sí	Sí	No	Parcial
ElParking	No	Sí	Sí	Sí	Sí	Parcial
SpotHero (US/Uber)	Parcial	Sí	Sí	Sí	Sí	No
JustPark (UK)	Sí	Sí	Sí	Sí	Parcial	No
ReservPark (ES)	Sí	Sí	Sí	Sí	No	No
SocialPark (ES)	Sí	Sí	No	Parcial	No	Beta
Parky	Sí	Sí	Sí	Sí	Sí	Sí

1.4. Tendencias del mercado

El problema del aparcamiento no es anecdótico ni local. Hay datos que lo cuantifican con precisión, y esos datos ayudan a calibrar tanto la magnitud de la ineficiencia como el tamaño de la oportunidad que Parky trata de resolver.

1.4.1. El mercado de aparcamiento en España

España tiene uno de los parques vehiculares más densos de Europa. Al cierre de 2025 circulaban en España más de 34,7 millones de vehículos asegurados [18] lo que hace un 2,2 % más que el año anterior y el mayor incremento del parque desde 2018 Figura 1.1, lo que sitúa al país entre los índices de motorización más altos de la Unión Europea, comparable con Italia y Alemania.

¹⁰**IGIC** (Impuesto General Indirecto Canario): Tributo indirecto propio de las Islas Canarias que sustituye funcionalmente al IVA peninsular. Su tipo general es del 7 % y el reducido del 3 %. El alquiler de plazas entre particulares en Canarias queda sujeto a este impuesto.

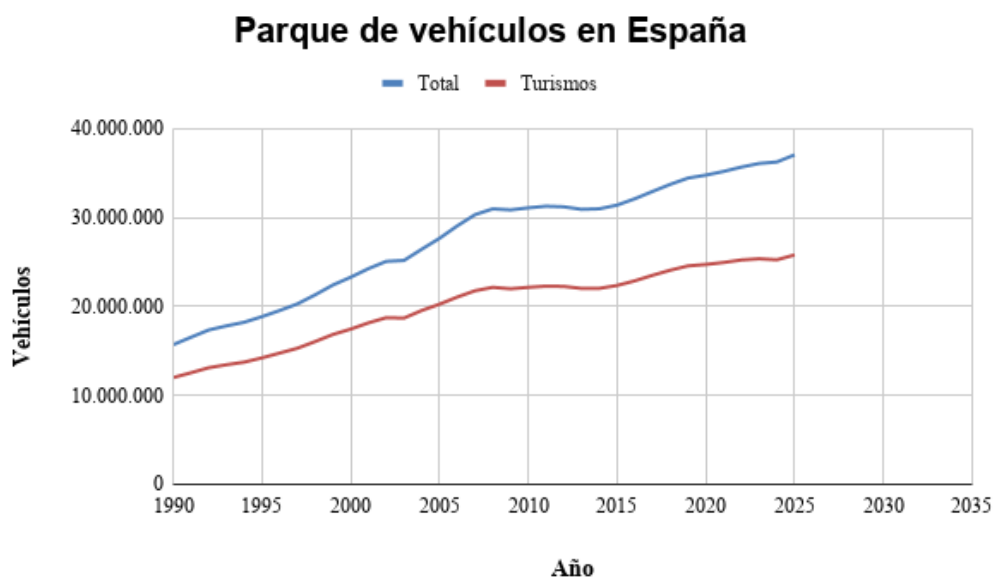


Figura 1.1: Evolución del parque de vehículos en España 1990–2025. Fuente: DGT [19].

Ese volumen tiene un coste real en tiempo. El informe anual de movilidad urbana de *INRIX* [4] estima que los conductores españoles dedican una media de 24 horas al año exclusivamente a buscar aparcamiento. En las grandes ciudades la cifra sube: Madrid, Barcelona y Valencia se sitúan entre las 20 capitales europeas con mayor tiempo de búsqueda. Por ejemplo, en Madrid, los conductores tardan entre 20 y 40 minutos de media en encontrar aparcamiento en zonas céntricas [20] haciendo que el tiempo en atasco sea de media unas 64 horas.

El aparcamiento regulado en calle y los garajes comerciales reflejan esa presión. En Santa Cruz de Tenerife, el nuevo sistema de zonas azul y verde previsto para 2026 [21] fija tarifas de 1 a 2 €/hora para no residentes en zona azul; los residentes pagarán menos de 1 €/hora o una cuota anual de unos 60 € en zona verde. En Madrid, el aparcamiento regulado de zona A (SER) alcanza 2,55 €/hora, pero un garaje comercial de centro puede superar los 4,50 €/hora o los 36 € por 24 horas [22]. En Barcelona, los garajes de máxima demanda superan los 5 €/hora, con jornadas completas en torno a los 30 € [22].

Tabla 1.2: Comparativa de tarifas de aparcamiento regulado por ciudad y zona.

Ciudad	Zona	Tarifa (€/hora)	Fuente
Santa Cruz de Tenerife	Zona azul (no res.)	1,00–2,00	Ayto. Santa Cruz 2026
Santa Cruz de Tenerife	Zona verde (res.)	<1,00 / 60 €/año	Ayto. Santa Cruz 2026
Madrid	Zona A (SER)	2,55	EMT / Ayto. Madrid
Madrid	Zona B (SER)	1,75	EMT / Ayto. Madrid
Barcelona	Zona 1 (àrea verda)	2,75	BSM / Ajto. Barcelona
Barcelona	Zona 2	1,40	BSM / Ajto. Barcelona

1.4.2. El mercado en Canarias y el área metropolitana de Tenerife

Canarias presenta una situación particular. La insularidad limita las alternativas de transporte público y convierte al vehículo privado en la opción dominante para la mayoría de desplazamientos. El Instituto Canario de Estadística [23] registra en Tenerife una tasa de 870 vehículos por cada 1.000 habitantes en 2025 [24] como se puede ver en la Figura 1.2 siendo esta la más alta del archipiélago, con una edad media del parque superior a los 27 años y más del 13 % de los vehículos superando los 30 años de antigüedad.

La presión sobre el aparcamiento en Santa Cruz de Tenerife y San Cristóbal de La Laguna se explica por la combinación de tres factores: alta densidad residencial en el casco histórico, escasa oferta de aparcamientos públicos subterráneos y concentración de actividad comercial y universitaria. Solo la Universidad de La Laguna, con más de 20.000 estudiantes matriculados, genera una demanda diaria de aparcamiento que actualmente no puede ser absorbida.

A ello se añade la presión normativa ya que la *Ley 7/2021*¹¹, de cambio climático y transición energética [25], obliga a los municipios de más de 50.000 habitantes a implantar las ZBE antes del 1 de enero de 2023. Aunque esto se ha ido posponiendo y se prevé que se acabe instaurando en la capital tinerfeña a principios de verano de 2026 haciendo mucho más complicado el acceso al centro urbano de los vehículos antiguos ya mencionados, lo que a su vez incrementa la demanda de aparcamiento en las zonas periféricas.

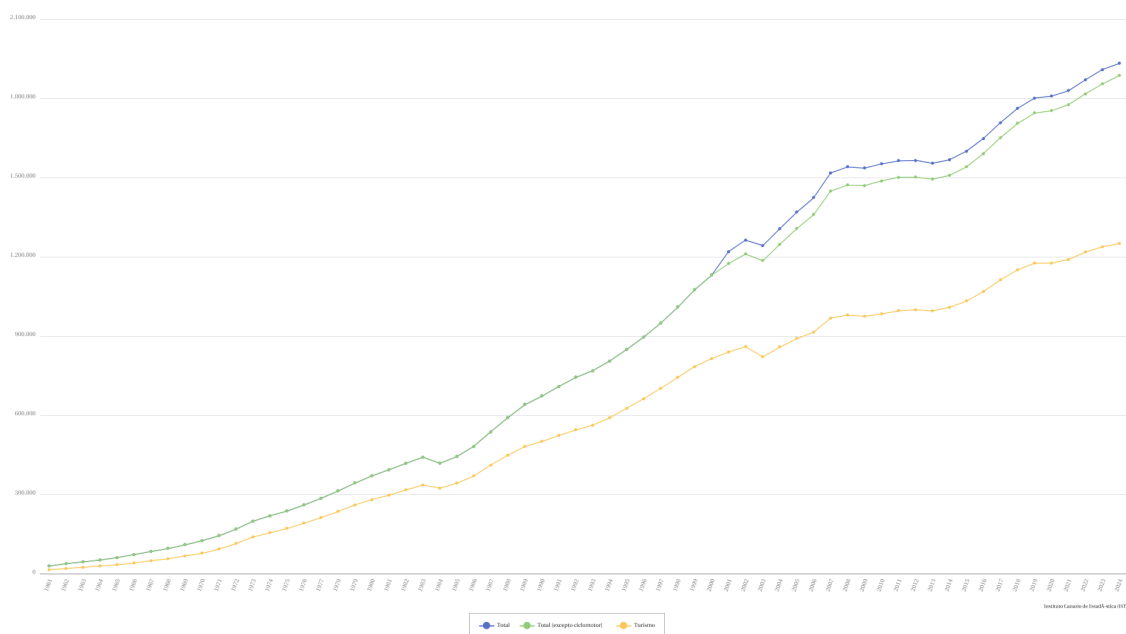


Figura 1.2: Evolución anual del parque de vehículos en Canarias (datos desde 1961). Fuente: IS-TAC [23].

¹¹**Ley 7/2021:** establece la obligatoriedad de las ZBE con carácter vinculante. Santa Cruz de Tenerife (aprox. 210.000 hab.) y San Cristóbal de La Laguna (aprox. 160.000 hab.) quedan dentro de su ámbito de aplicación. La restricción de acceso vehicular no elimina la necesidad de aparcar: la desplaza hacia la periferia, donde la oferta de plazas privadas infrautilizadas es mayor.

1.4.3. La Zona de Bajas Emisiones de Santa Cruz de Tenerife

El 13 de abril de 2026, el Ayuntamiento de Santa Cruz de Tenerife aprobó el borrador definitivo de la Ordenanza de Movilidad Sostenible que establece la ZBE municipal [26]. La zona abarca 778.244 m² del casco histórico y el distrito Centro, delimitada por la calle Méndez Núñez, la Rambla de Santa Cruz, la avenida de Anaga y el barranco de Santos.

Las restricciones operan de lunes a sábado, de 07:00 a 20:00 horas. Se vetan los vehículos sin etiqueta ambiental DGT: turismos de gasolina matriculados antes de 2001 y diésel anteriores a 2006. La ordenanza prevé 18 meses de carencia para el registro de vehículos y no aplica sanciones efectivas (200 €) hasta 2029.

La normativa contempla una excepción relevante para Parky. Los vehículos con reserva activa en un aparcamiento privado dentro del perímetro quedan exentos de las restricciones ordinarias de acceso. Esta exención convierte a la plataforma en un habilitador de entrada legal al centro urbano, a la vez que redirige la demanda de estacionamiento hacia plazas privadas perimetrales.

1.4.4. Disponibilidad y precios en el modelo P2P

El alquiler de plazas de garaje en Santa Cruz de Tenerife oscila entre 50 y 150 €/mes según ubicación, con la mayor concentración de oferta en el tramo de 60–100 € [27] frente a los 7–15 €/día de un parking comercial de rotación. Incluso la tarifa mensual más alta de alquiler resulta competitiva para un conductor que necesita aparcar a diario.

Para el propietario, la relación es igualmente favorable. Una plaza que genera 80 €/mes suma 960 € anuales por un activo que de otro modo permanece vacío. En mercados donde el modelo P2P ya funciona a escala, JustPark [13] documenta entre 500 y 3.000 libras anuales por espacio publicado según la ubicación, lo que da una referencia del potencial del modelo.

La restricción principal del modelo P2P no es de precio, sino de oferta organizada. **No existe en Canarias ninguna plataforma que agregue esas plazas dispersas, las haga visibles en un mapa y gestione la reserva y el acceso de forma automática.** Esa ausencia define el nicho que Parky busca cubrir.

1.5. Objetivos

Objetivo general

El objetivo general es diseñar y desarrollar un prototipo funcional de *marketplace P2P* para el alquiler de plazas de aparcamiento privadas en el área metropolitana de Tenerife, que conecte a propietarios de garajes infrautilizados con conductores que necesitan estacionar, y que resuelva de forma integrada la gestión de reservas, el acceso físico al garaje y el flujo de pagos entre particulares.

Objetivos específicos

- OE1.** Implementar un sistema de autenticación y control de acceso basado en roles (RBAC)¹² que diferencie la lógica de permisos entre arrendatarios, propietarios y administradores, con soporte para inicio de sesión mediante proveedores externos (OAuth)¹³.
- OE2.** Construir una *Single Page Application*¹⁴ con enfoque *mobile-first* (ya que la mayoría de reservas se realizan desde móviles) que integre búsqueda sobre mapa interactivo, filtros bidireccionales en tiempo real y geocodificación para el registro de nuevas plazas de aparcamiento.
- OE3.** Garantizar el aislamiento de datos entre usuarios mediante políticas de seguridad (*Row Level Security*, RLS)¹⁵ aplicadas directamente en el esquema relacional de PostgreSQL.
- OE4.** Prevenir solapamientos en reservas concurrentes mediante *triggers* y procedimientos almacenados en PL/pgSQL, con validación de disponibilidad de plaza ejecutada íntegramente en base de datos, independiente del cliente que origine la petición.
- OE5.** Implementar el flujo de *checkout*, separando automáticamente la tarifa base del arrendador, la comisión de la plataforma y el IGIC del 7 % correspondiente a la jurisdicción canaria, y liquidando los pagos directamente a la cuenta bancaria del propietario.
- OE6.** Implementar los estados de reservas mediante rutinas programadas en base a la hora y día actuales, el módulo SmartAccess de apertura remota de garajes con trazabilidad de accesos, y un sistema de valoraciones bidireccionales entre arrendatario y propietario que incentive la confianza en el intercambio entre particulares.
- OE7.** Desplegar la aplicación como cliente web y como aplicación Android nativa desde una base de código TypeScript compartida, utilizando Capacitor¹⁶ como puente entre la capa web y el sistema operativo móvil.

¹²**RBAC** (*Role-Based Access Control*): modelo que asigna permisos a los usuarios según el rol que desempeñan, sin conceder permisos individuales a cada cuenta.

¹³**OAuth** (*Open Authorization*): protocolo que permite a aplicaciones externas acceder a recursos de un usuario mediante tokens de acceso, sin que este comparta sus credenciales directamente.

¹⁴**SPA** (*Single Page Application*): aplicación web que carga un único documento HTML y actualiza su contenido dinámicamente mediante JavaScript, sin recargar la página desde el servidor.

¹⁵**RLS** (*Row Level Security*): mecanismo de PostgreSQL que restringe el acceso a filas de una tabla según políticas definidas en el esquema, sin depender de filtros en la capa de aplicación.

¹⁶**Capacitor**: herramienta de Ionic que empaqueta una aplicación web en un contenedor nativo iOS/Android, dando acceso a las APIs del dispositivo con un único código fuente.

Capítulo 2

Estudio de tecnologías y decisiones de diseño

Este capítulo describe las tecnologías que componen Parky y justifica cada elección frente a las alternativas consideradas. Para cada herramienta se explica qué función cumple en el sistema, por qué se prefirió sobre las alternativas consideradas y qué compromisos implica la elección.

2.1. Lenguajes de programación y frameworks

La arquitectura separa con claridad dos ámbitos: el *frontend*, ejecutado en el navegador o en el dispositivo Android, y el *backend*, delegado íntegramente a servicios externos gestionados. Esta separación permitió concentrar el desarrollo en la lógica de negocio y la experiencia de usuario sin gestionar servidores propios.

2.1.1. Frontend

El frontend de Parky se organiza en tres capas funcionales: el lenguaje y el sistema de tipos, el framework de renderizado con su cadena de herramientas, y la capa de gestión de datos, formularios y estilos.

Lenguaje y tipado. Todo el código fuente está escrito en *TypeScript 5* [28] porque su compilador actúa como primera línea de defensa: detecta incompatibilidades de tipo, propiedades inexistentes y posibles valores nulos antes de que el código llegue al navegador. Esta garantía es especialmente valiosa en Parky, donde los errores de tipo tienen consecuencias directas sobre la integridad del negocio. Por ejemplo, una validación incorrecta en el intervalo de reserva puede crear solapamientos, y un error en el desglose de precio base, comisión e IGIC puede generar cargos incorrectos en Stripe. Los componentes de React se expresan en *JSX*¹, y *SWC*² se encarga de transformar ese código a JavaScript puro antes de enviarlo al navegador.

¹**JSX** (*JavaScript XML*): Extensión de sintaxis que permite escribir marcado similar a HTML dentro del código JavaScript.

²**SWC** (*Speedy Web Compiler*): Transpilador escrito en Rust que reemplaza a Babel. Es entre 20 y 70 veces más rápido en compilaciones en frío.

Framework y renderizado. La interfaz se construye con *React 18.3.1* [29]. El argumento determinante para elegir esta versión fue el *modo concurrente*: características como `useTransition` y `startTransition` permiten marcar como no urgentes las actualizaciones costosas en componentes React —por ejemplo, refrescar el listado de plazas disponibles al mover el mapa— de modo que el motor puede aplazar esa actualización para atender primero los gestos táctiles del usuario, evitando que los paneles de filtros o resultados bloqueen la interacción durante cargas intensivas.

El empaquetador es *Vite 6.4.1* [30], que acelera el ciclo de desarrollo al evitar recompilar toda la aplicación ante cada cambio, y optimiza el código para producción eliminando automáticamente las partes no utilizadas. El enrutado usa *React Router DOM 7* [31], configurado con *lazy loading* para cargar el código de cada pantalla únicamente cuando el usuario la visita, lo que reduce el tiempo de carga inicial de la aplicación.

El factor que descartó las alternativas fue la integración con *Capacitor*, el adaptador que empaqueta la aplicación web como APK nativo de Android. Angular quedó descartado por su curva de aprendizaje y menor afinidad con Capacitor; Vue y Svelte ofrecen mejoras de rendimiento notables pero su ecosistema para integraciones híbridas móviles es mucho más reducido. La Tabla 2.1 resume los criterios decisivos:

Tabla 2.1: Comparativa de frameworks JavaScript de frontend para el desarrollo de Parky.

Criterio	React 18	Vue 3	Angular 17	Svelte 4
Modo concurrente	Sí (v18)	No	No	No
App nativa (Capacitor)	Completa	Limitada	Limitada	Experimental
Ecosistema híbrido móvil	Muy amplio	Reducido	Reducido	Muy reducido

Estado, formularios y estilos. La sincronización de datos con el servidor se gestiona con *TanStack Query v5* [32]³, que mantiene la interfaz actualizada automáticamente sin que el desarrollador gestione manualmente cuándo y cómo refrescar los datos. Esto es especialmente relevante en Parky, donde la disponibilidad de una plaza puede cambiar en cualquier momento por una reserva concurrente.

Los formularios (alta de plaza, inicio de sesión, selección de intervalo de reserva) se gestionan con *React Hook Form 7*⁴. A diferencia del enfoque estándar de React, los campos no provocan un re-render de la interfaz en cada pulsación de teclado, lo que importa especialmente en formularios con validaciones cruzadas como el selector de intervalo horario de una reserva.

El sistema de estilos usa *Tailwind CSS 4.1.3* [33]⁵. Los componentes de interfaz se construyen sobre *Radix UI*⁶, que garantiza un comportamiento accesible por defecto (gestión de foco, navegación por

³**TanStack Query**: biblioteca de gestión de estado asíncrono que abstrae la caché, el ciclo de vida y la sincronización de datos remotos en aplicaciones React.

⁴**React Hook Form**: biblioteca de gestión de formularios que registra los campos mediante refs en lugar de estado de React, minimizando los re-renders durante la edición.

⁵**Tailwind CSS**: framework de estilos utility-first que compone el diseño visual mediante clases atómicas aplicadas directamente en el marcado, en lugar de hojas CSS por componente.

⁶**Radix UI**: biblioteca de componentes primitivos para React que garantiza accesibilidad por defecto sin imponer estilos propios.

teclado, atributos ARIA, etc.) de modo que Tailwind controla completamente la apariencia visual sin fricciones.

2.1.2. APIs externas de cartografía y geocodificación

En Parky la localización geográfica no es un complemento sino el núcleo de la experiencia: el arrendatario busca plazas dentro del área visible del mapa, las visualiza como marcadores geoposicionados y accede a la dirección exacta del garaje. Esto exige dos capacidades independientes: renderizar mapas interactivos y convertir direcciones de texto en coordenadas geográficas.

Leaflet. La visualización cartográfica usa *Leaflet 1.9.4* [34] con el adaptador *React-Leaflet 4.2.1*. La alternativa natural era la API de *Google Maps*, descartada por su modelo de facturación por volumen. Leaflet es *open source* (bajo licencia BSD-2), sin coste por carga de mapa, y su rendimiento en dispositivos Android de gama media es suficiente para los casos de uso de Parky.

OpenCage Geocoding API. La conversión entre direcciones de texto y coordenadas geográficas usa *OpenCage Geocoding API* [35]. Este servicio agrega datos de OpenStreetMap, Geonames y otras fuentes abiertas con soporte para direcciones canarias, lo que es fundamental para la aplicación. El plan gratuito incluye 2.500 peticiones diarias, suficiente para el volumen esperado durante la fase de prototipado.

2.1.3. Backend

Se evaluó desarrollar un servidor a medida con *Node.js* o *Python* frente al uso de plataformas *BaaS*⁷, pero se optó por *Supabase* [36] por tres razones.

Supabase cuenta con una capa de autenticación que gestiona el cifrado de contraseñas, los flujos OAuth (Google) de forma nativa y las sesiones mediante *JWT*⁸, eliminando la necesidad de implementar y auditar un sistema de identidades propio. También cuenta con un almacenamiento de objetos que gestiona las imágenes de garajes y perfiles con políticas de acceso integradas que impiden que un usuario modifique recursos ajenos. Por último y más importante, para los pagos, la plataforma cuenta con integración con *Stripe Connect*⁹ por medio de Edge Functions, que calculan la comisión de la plataforma, el *IGIC* y transfieren el saldo neto directamente a la cuenta bancaria del propietario del garaje sin que los fondos pasen por una cuenta bancaria central, lo que simplifica la fiscalidad.

Un desarrollo backend propio habría requerido implementar estos tres sistemas desde cero, con la carga de mantenimiento y la superficie de ataque que eso implica en un equipo de una sola persona.

⁷**BaaS (Backend-as-a-Service):** Modelo donde un proveedor externo aloja la infraestructura central del servidor —almacenamiento, autenticación, base de datos— y el desarrollador la consume a través de APIs documentadas.

⁸**JWT (JSON Web Token):** estándar RFC 7519 que define un formato compacto para transmitir información entre partes como objeto JSON firmado criptográficamente.

⁹**Stripe Connect:** arquitectura de pagos de Stripe para *marketplaces* que separa y liquida automáticamente los fondos entre las partes.

2.1.4. Base de datos

Se compararon el modelo relacional y el modelo documental para la capa de persistencia. El dominio de Parky es inherentemente relacional: una reserva vincula a un usuario con una plaza, que pertenece a un garaje, durante un intervalo de tiempo con un pago asociado. *MongoDB* [37] exigiría replicar esa cadena en múltiples documentos, con el riesgo de inconsistencia ante una operación fallida a mitad de ejecución. Es por eso que se eligió *PostgreSQL* [38] porque resuelve esa inconsistencia con integridad referencial mediante *foreign keys*¹⁰ y *transacciones ACID*¹¹.

Dentro de *PostgreSQL 15* se usan dos características avanzadas: las *políticas RLS* que garantizan que cada usuario solo pueda leer sus propias reservas y los *triggers PL/pgSQL* que comprueban, antes de confirmar cualquier reserva, que no exista solapamiento horario en la misma plaza, manteniendo la consistencia incluso ante peticiones simultáneas.

2.2. Entorno de desarrollo

Editor de código. El entorno de desarrollo es *Visual Studio Code* [39]. Frente a IDEs¹² basados en IntelliJ, su consumo de memoria es sensiblemente menor, lo que resulta determinante en el desarrollo híbrido con Capacitor, donde el emulador de Android debe estar abierto en paralelo. Además, integra de forma nativa el servidor de TypeScript, por lo que el análisis de tipos en el editor es idéntico al de la compilación real.

Control de versiones. El control de versiones usa *Git* [40] con *GitHub* [41] como repositorio remoto. Esta combinación no solo documenta el historial de cambios por fases de desarrollo, sino que actúa como punto de integración con el pipeline de despliegue continuo de Vercel: cada commit en la rama principal desencadena automáticamente un nuevo despliegue en producción.

Diseño de interfaces. El diseño visual previo a la implementación se realizó con *Figma* [42], que permitió validar la disposición de pantallas, la paleta de colores y las distancias entre elementos antes de traducirlos a clases de Tailwind CSS. Trabajar con un prototipo de alta fidelidad redujo el número de decisiones de diseño durante el desarrollo, concentrando ese esfuerzo en una fase anterior donde el coste de cambio es menor. La figura 2.1 muestra la evolución entre el prototipo inicial y la pantalla principal en producción.

¹⁰**Foreign key** (clave foránea): columna que referencia la clave primaria de otra tabla, garantizando que no puedan existir reservas de plazas inexistentes o registros huérfanos.

¹¹**ACID**: propiedades de Atomicidad, Consistencia, Aislamiento y Durabilidad que garantizan que las operaciones de base de datos se ejecutan completamente o no se ejecutan en absoluto.

¹²**IDE** (*Integrated Development Environment*): entorno que agrupa editor, compilador y depurador en una única aplicación.

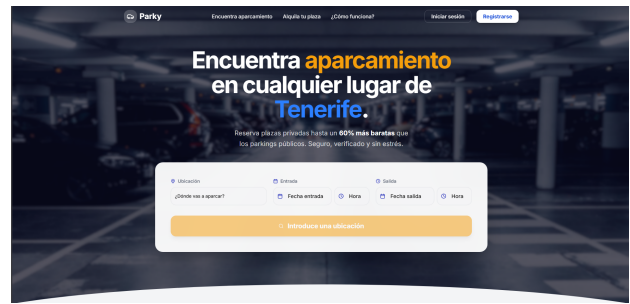


Figura 2.1: Comparativa entre el prototipo en Figma (izquierda) y la pantalla principal de Parky en producción (derecha).

2.3. Plataformas de despliegue

Web: Vercel. La versión web se despliega en *Vercel* [43], elegido por su integración nativa con GitHub ya que cada commit en la rama principal desencadena automáticamente una nueva compilación y distribución en su CDN global, sin configuración adicional. Gestiona además los certificados TLS de forma automática. El dominio público de la página se registró en One.com [44] y se enlazó a Vercel mediante registros DNS.

Android: Capacitor. La aplicación Android se genera con *Capacitor 8.1.0* [45], que encapsula la aplicación web dentro de una WebView nativa. A diferencia de React Native, que requiere reescribir los componentes en primitivas nativas, Capacitor reutiliza las vistas TypeScript/HTML sin modificaciones, lo que permite mantener una única base de código para web y Android. El APK final se compila desde Android Studio sin modificar ningún componente de React.

2.4. Análisis de alternativas y decisiones de diseño

La tabla 2.2 recoge la comparativa de plataformas BaaS que determinó la elección de Supabase como infraestructura central del proyecto.

Tanto Firebase como AWS Amplify están pensados para modelos de datos documentales y escalan bien en ese contexto, pero el dominio de Parky no encaja ahí. La cadena reserva — usuario — plaza — garaje — intervalo — pago se expresa de forma natural en tablas relacionadas mediante JOINs; trasladarla a Firestore obligaría a replicar datos en varios documentos, lo que introduce riesgo de inconsistencia ante cualquier escritura parcial fallida. AWS Amplify ofrece Aurora Serverless como salida relacional, pero su configuración es considerablemente más compleja y carece de RLS nativo, lo que habría obligado a gestionar el aislamiento de datos entre roles desde la capa de aplicación en lugar de resolverlo directamente en el esquema de base de datos.

El principal compromiso de Supabase es su plan gratuito: 500 MB de almacenamiento y pausa automática del proyecto tras 7 días de inactividad. Para un prototipo universitario ambos límites son asumibles, pero una versión comercial requeriría migrar al plan Pro (\$25/mes).

Tabla 2.2: Comparativa de plataformas BaaS frente a las necesidades de Parky.

Característica	Supabase [36]	Firebase [46]	AWS Amplify [47]
Motor de base de datos	PostgreSQL (relacional)	Firestore (documental)	DynamoDB / Aurora Serverless
Consultas relacionales	SQL nativo con JOINS	Limitadas, sin JOINS	Posibles con Aurora Serverless
Row Level Security	Nativo en PostgreSQL	No disponible	No disponible
Autenticación	Nativo (email, OAuth, MFA)	Authentication integrado	Amazon Cognito
Triggers y funciones	PL/pgSQL + Edge Functions (Deno)	Cloud Functions (externo)	Lambda (externo)
Modelo de datos	Normalizado, tipado	Flexible, sin esquema	Mixto (NoSQL o relacional)
Límite tier gratuito	500 MB / 2 proyectos	1 GB (Spark Plan)	12 meses AWS Free Tier

Capítulo 3

Metodología y análisis de requisitos

Este capítulo establece el marco metodológico del proyecto y formaliza los actores del sistema, los requisitos funcionales y no funcionales, los casos de uso y el diseño de la base de datos.

3.1. Metodología de desarrollo

El *modelo en cascada*¹ fue descartado desde el inicio por una razón estructural importantísima: en un sistema con múltiples capas interdependientes (autenticación, geolocalización, pagos y aplicación nativa) la suposición de que todos los requisitos pueden conocerse antes de implementar es falsa. Especificar completamente el sistema antes de construirlo habría sido imposible en Parky, y tres episodios durante el desarrollo lo demuestran.

En primer lugar, el módulo *SmartAccess* no figuraba en el análisis inicial y emergió en el sprint 4 porque al construir el flujo de reservas quedó claro que «el arrendatario llega al garaje y no puede entrar» era exactamente la fricción central que la plataforma pretendía eliminar. En segundo lugar, el *trigger* de solapamiento de reservas se añadió en el sprint 8, no al principio ya que solo al implementar el flujo completo se hizo evidente que dos usuarios podían reservar la misma plaza simultáneamente. En tercer lugar, la auditoría de seguridad del sprint 7 detectó que el diseño de borrado de reservas destruía el historial financiero de los arrendadores, algo que solo resulta visible cuando el sistema maneja datos reales. En cascada, cualquiera de estos tres hallazgos habría obligado a reabrir la fase de requisitos y rediseñar desde arriba, con un coste de retroceso enorme.

Por ello se adoptó **Scrum** [48]² con *sprints*³ de dos semanas, cada uno centrado en un módulo de alto nivel. Scrum acepta que los requisitos evolucionan y lo convierte en una ventaja en lugar de un problema de modo que los errores que se detectan al final de cada ciclo, se corrigen en el siguiente sin afectar lo ya construido. Al tratarse de un proyecto unipersonal, los tres roles canónicos del marco (*Product Owner*, *Scrum Master* y equipo de desarrollo) convergieron en una única persona, y la retrospectiva de cada ciclo fue el mecanismo principal para detectar *deuda técnica*⁴ antes de que se acumulara.

La Tabla 3.1 recoge los nueve sprints en los que se desarrolló el prototipo, con sus respectivos módulos principales y entregables.

¹**Modelo en cascada:** Modelo de desarrollo secuencial en el que cada fase (análisis, diseño, desarrollo, pruebas y despliegue) debe cerrarse completamente antes de iniciar la siguiente. Su principal limitación es que asume que los requisitos pueden especificarse en su totalidad antes de escribir una sola línea de código.

²**Scrum:** Marco de trabajo ágil definido por Schwaber y Sutherland [48] que organiza el desarrollo en iteraciones de duración fija llamadas *sprints*, al término de cada una de las cuales se entrega un incremento funcional del producto.

³**Sprints:** Iteración de duración fija (dos semanas en Parky) al final de la cual se revisa el trabajo entregado, se ajusta el *backlog* y se reprioriza el siguiente ciclo.

⁴**Deuda técnica:** Coste futuro de corrección acumulado al tomar decisiones de implementación rápidas en lugar de óptimas. En un proyecto unipersonal, la retrospectiva de sprint es el único mecanismo formal para identificarla antes de que se propague.

Tabla 3.1: Sprints de desarrollo de Parky (enero–mayo de 2026).

Sprint	Período	Módulo principal	Entregable
1	6–19 ene	Infraestructura base	Scaffolding React/Vite, autenticación Supabase, rutas protegidas y modo oscuro.
2	20 ene–2 feb	Modelo de datos y perfil	Esquema de base de datos, RLS básico, edición de perfil e imágenes en Storage.
3	3–16 feb	Plazas y mapa	Integración Leaflet, CRUD de garajes y plazas, React Query como capa de caché.
4	17 feb–2 mar	Reservas, SmartAccess y pagos	Motor de precios, módulo SmartAccess con log de accesos y arquitectura de pagos.
5	3–16 mar	Despliegue multiplataforma	Vercel, compilación Android con Capacitor y APK nativo.
6	17 mar–26 abr	Memoria del TFG	Redacción de los capítulos 1 a 4; commits de código suspendidos.
7	27 abr–10 may	Seguridad y calidad	Auditoría de RLS, borrado lógico en bookings y HSTS/CSP en Vercel.
8	10–14 may	Disponibilidad y reserva	Horarios recurrentes, trigger de solapamiento y buffer de 15 minutos.
9	15–19 may	Pulido UX	<i>Code splitting</i> (reducción del 95 % del <i>bundle</i>), navegación inferior y <i>touch targets</i> móvil.

3.2. Actores del sistema

En cuanto a los actores del sistema, Parky define cuatro roles principales definidos con claridad en la tabla 3.2. La separación entre arrendador y arrendatario no es solo conceptual sino también funcional, porque cada rol queda restringido a un subconjunto de datos distinto impuesto por las políticas RLS de la base de datos.

Tabla 3.2: Actores del sistema Parky.

Actor	Responsabilidades y alcance
Arrendador	Propietario de garajes. Publica plazas con precio, fotos y geolocalización. Acepta o rechaza reservas entrantes. Recibe pagos a través de Stripe Connect.
Arrendatario	Conductor que alquila plazas. Busca en el mapa, reserva, paga con Stripe y valora la plaza al finalizar.
Administrador	Opera desde el <i>dashboard</i> de Supabase con privilegios de servicio. Suspende cuentas, corrige datos y ejecuta consultas de mantenimiento. Sin interfaz en el cliente web.
Sistema	Componente automatizado. Procesa transacciones (Stripe), geocodifica direcciones (OpenCage), gestiona sesiones (OAuth) y emite notificaciones almacenadas en base de datos.

3.3. Requisitos funcionales

Los requisitos funcionales se construyeron a partir de los objetivos específicos OE1–OE7 definidos en el capítulo introductorio dentro de la sección 1.5 y de un análisis de las funcionalidades de plataformas P2P de referencia. El prototipo final cubre los dieciséis requisitos funcionales listados en la Tabla 3.3, con la salvedad de que RF-08 y RF-16 están implementados a nivel de arquitectura y modelo de datos pero sin procesar cobros reales, al tratarse de un prototipo académico y que no es necesario el cobro para mostrar la funcionalidad. Los dieciséis requisitos funcionales se agrupan por módulo en la Tabla 3.3:

3.4. Requisitos no funcionales

Los requisitos no funcionales establecen los criterios de calidad que el sistema debe satisfacer con independencia de las funcionalidades concretas, y se pueden ver en la tabla 3.4. El criterio de rendimiento responde a un dato práctico, ya que los usuarios de Parky consultan el mapa mientras circulan por la ciudad y una respuesta superior a tres segundos en condiciones de conectividad habituales provoca el abandono antes de completar la búsqueda. El diseño se adapta a resoluciones de gama media porque ese es el perfil de dispositivo predominante entre los conductores que buscan aparcamiento en Tenerife, y la misma base de código React compila sin modificaciones como aplicación nativa de Android, lo que cubre el requisito de portabilidad sin duplicar el desarrollo. La disponibilidad del servicio se apoya en las garantías de Vercel y Supabase, cuya infraestructura redundante sería inviable de replicar con un servidor propio a este coste de proyecto.

Tabla 3.3: Requisitos funcionales de Parky.

ID	Actor	Descripción
RF-01	Arr. / Arrend. / Admin.	Registro e inicio de sesión mediante email/contraseña y OAuth (Google).
RF-02	Arrendador / Arrendatario	El sistema asigna automáticamente el rol <i>Arrendatario</i> al registrar al usuario; al crear su primer garaje, el perfil adquiere también el rol <i>Arrendador</i> , quedando ambos activos simultáneamente en la misma cuenta.
RF-03	Arrendador	Dar de alta un garaje con nombre, dirección y fotos, y gestionar sus plazas individuales (crear, editar, desactivar) fijando precio por hora y disponibilidad.
RF-04	Arrendatario	Visualizar plazas disponibles en el mapa dentro del área visible y filtrarlas por precio y horario.
RF-05	Arrendatario	Solicitar reserva sobre una plaza con fecha y hora de inicio y fin.
RF-06	Sistema	Rechazar reservas que solapen con otra confirmada o en curso en la misma plaza.
RF-07	Arrendador	Las reservas entrantes permanecen en estado pendiente hasta su confirmación o rechazo manual.
RF-08	Sistema	Calcular el importe del cobro con desglose de comisión de plataforma e IGIC (7 %), y registrar la transacción en el modelo de datos de pagos. La integración con la API de Stripe Connect para procesar cobros reales está prevista para la versión comercial (véase <i>roadmap</i>).
RF-09	Sistema	Reembolsar íntegramente al arrendatario si la reserva se cancela antes del inicio.
RF-10	Arrendatario	Valorar una plaza de 1 a 5 únicamente si la reserva está en estado completada.
RF-11	Sistema	Notificar al arrendador cuando llegue una nueva solicitud de reserva.
RF-12	Sistema	Notificar al arrendatario cuando el arrendador confirme o rechace su reserva.
RF-13	Arrendador / Arrendatario	Editar nombre, fotografía de perfil y datos de contacto propios.
RF-14	Arrendador / Arrendatario	Consultar el historial de reservas propias con estado, fecha e importe.
RF-15	Sistema	Generar un código QR único para cada reserva confirmada, válido únicamente durante la ventana temporal activa (SmartAccess).
RF-16	Arrendador	Completar el alta en Stripe Connect para vincular una cuenta bancaria y habilitar la recepción de transferencias. Previsto para la versión comercial.

Tabla 3.4: Requisitos no funcionales de Parky.

Categoría	Criterio
Rendimiento	LCP^5 inferior a 3 s bajo conectividad 4G de latencia media.
Seguridad	Todas las tablas tienen RLS activo. Cada petición al backend se valida con JWT. Ninguna clave secreta se expone en el cliente.
Usabilidad	Diseño <i>mobile-first</i> con punto de ruptura desde 375 px.
Portabilidad	La base de código React compila como APK para Android mediante Capacitor sin modificar componentes.
Disponibilidad	Vercel y Supabase garantizan un <i>uptime</i> del 99,5 % en sus SLA.
Mantenibilidad	El código fuente sigue convenciones de tipado estrictas verificadas en cada compilación, lo que reduce la probabilidad de errores silenciosos antes de llegar a producción.

3.5. Modelo de casos de uso

El modelo de casos de uso describe las interacciones entre los cuatro actores del sistema y las funcionalidades que Parky expone. El diagrama de la Figura 3.1 organiza estos casos de uso por módulo funcional, y las tablas siguientes detallan los cuatro de mayor peso con sus precondiciones, postcondiciones y flujos alternativos.

Tabla 3.5: CU-01: Autenticarse y obtener rol.

ID y Nombre	CU-01: Autenticarse y obtener rol
Actor	Todos
Precondición	Sin sesión activa.
Postcondición	JWT activo y rol almacenado en <i>profiles</i> .
Flujo principal	1. El usuario accede a la pantalla de autenticación. 2. Se autentica con email/contraseña o Google OAuth. 3. El sistema asigna automáticamente el rol Arrendatario; si el usuario crea un garaje posteriormente, el perfil asciende a Arrendador sin intervención manual. 4. El rol se persiste en <i>profiles</i>. 5. El sistema redirige al <i>dashboard</i> correspondiente.
Flujo alternativo	Si el proveedor OAuth devuelve error, se ofrece reintento o autenticación con email/contraseña.

Tabla 3.6: CU-02: Publicar plaza de garaje.

ID y Nombre	CU-02: Publicar plaza de garaje
Actor	Arrendador
Precondición	Sesión activa con rol arrendador.
Postcondición	Plaza registrada y visible en el mapa.
Flujo principal	1. El arrendador accede al formulario de alta de plaza. 2. Introduce descripción, precio por hora y fotos. 3. Confirma la posición geográfica sobre el mapa interactivo. 4. El cliente ejecuta una inserción en la API REST de Supabase. 5. El sistema confirma el alta y actualiza el mapa.
Flujo alternativo	Si la subida de imágenes al <i>bucket</i> de Supabase Storage falla, el registro no se persiste y se notifica al usuario.

Tabla 3.7: CU-03: Buscar y filtrar plazas en el mapa.

ID y Nombre	CU-03: Buscar y filtrar plazas
Actor	Arrendatario
Precondición	Sesión activa y permisos de geolocalización concedidos.
Postcondición	Mapa con marcadores de plazas activas en el área visible.
Flujo principal	1. La aplicación obtiene la posición GPS del dispositivo. 2. Se consultan plazas en estado activo dentro del bounding box visible. 3. El usuario filtra por precio máximo o rango horario. 4. Al pulsar un marcador se muestra la ficha con precio y disponibilidad.
Flujo alternativo	Sin GPS disponible, el usuario introduce una dirección que OpenCage Geocoding convierte en coordenadas.

Tabla 3.8: CU-04: Realizar reserva y pago.

ID y Nombre	CU-04: Realizar reserva y pago
Actor	Arrendatario
Precondición	Sesión activa y plaza seleccionada.
Postcondición	Reserva en estado pendiente; intervalo horario bloqueado.
Flujo principal	1. El arrendatario selecciona fecha, hora de inicio y hora de fin. 2. El trigger PL/pgSQL verifica que no exista solapamiento en la misma plaza. 3. El usuario introduce los datos de tarjeta en el formulario de Stripe. 4. Stripe confirma el cargo y la reserva avanza a estado pendiente. 5. Ambas partes reciben notificación del cambio de estado.
Flujo alternativo	Si la tarjeta es rechazada o el pago expira, la reserva no se persiste y la plaza permanece disponible.

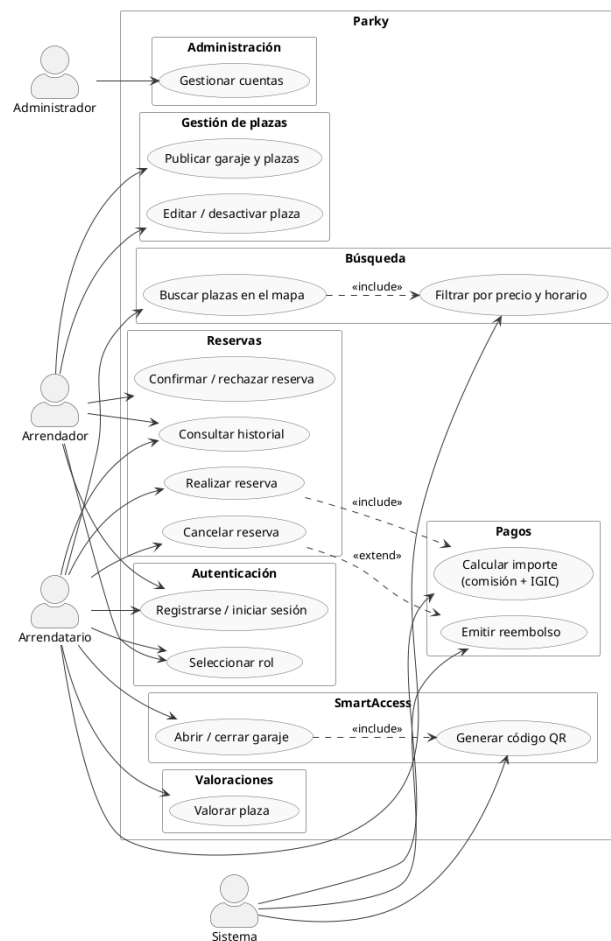


Figura 3.1: Diagrama de casos de uso del sistema Parky.

3.6. Trazabilidad de objetivos

La Tabla 3.9 recoge la correspondencia entre los objetivos específicos de la sección 1.5, los requisitos funcionales de la Tabla 3.3 y los casos de uso de la sección 3.5, mostrando que todos los objetivos quedan cubiertos por al menos un requisito y un caso de uso verificable.

Tabla 3.9: Matriz de trazabilidad: objetivo específico → req. funcional → caso de uso.

Objetivo	Req. funcionales	Casos de uso
OE1 — Autenticación y RBAC	RF-01, RF-02	CU-01
OE2 — SPA, mapa y geocodificación	RF-03, RF-04, RF-13	CU-02, CU-03
OE3 — Aislamiento RLS	RF-03 a RF-15 (operaciones sobre datos propios de cada rol)	CU-01 a CU-04
OE4 — Integridad de reservas	RF-05, RF-06	CU-04
OE5 — <i>Checkout</i> e IGIC	RF-08, RF-09	CU-04
OE6 — Estados, SmartAccess y valoraciones	RF-07, RF-10, RF-11, RF-12, RF-14, RF-15	CU-04
OE7 — Despliegue web y Android	RF-01 a RF-16	CU-01 a CU-04

3.7. Diseño de la base de datos

La base de datos se ejecuta sobre PostgreSQL 15 en Supabase. El esquema normaliza cada entidad de negocio en tablas independientes vinculadas por *foreign keys*. La tabla *profiles* almacena la información de cada usuario, incluyendo su rol (arrendador o arrendatario) y datos de contacto. La tabla *garages* representa los garajes publicados por los arrendadores, con su dirección y geolocalización. Cada garaje puede tener varias plazas, por lo que se almacenan en la tabla *parking_spots*, que incluye precio por hora y disponibilidad y se relaciona con *garages* mediante una *foreign key*. Las reservas se registran en la tabla *bookings* con su estado y ventana temporal, mientras que las valoraciones de los arrendatarios se almacenan en la tabla *reviews*. El modelo de pagos se apoya en la tabla *transactions*, que registra el importe, comisión e IGIC de cada reserva confirmada.

3.7.1. Diagrama de la base de datos

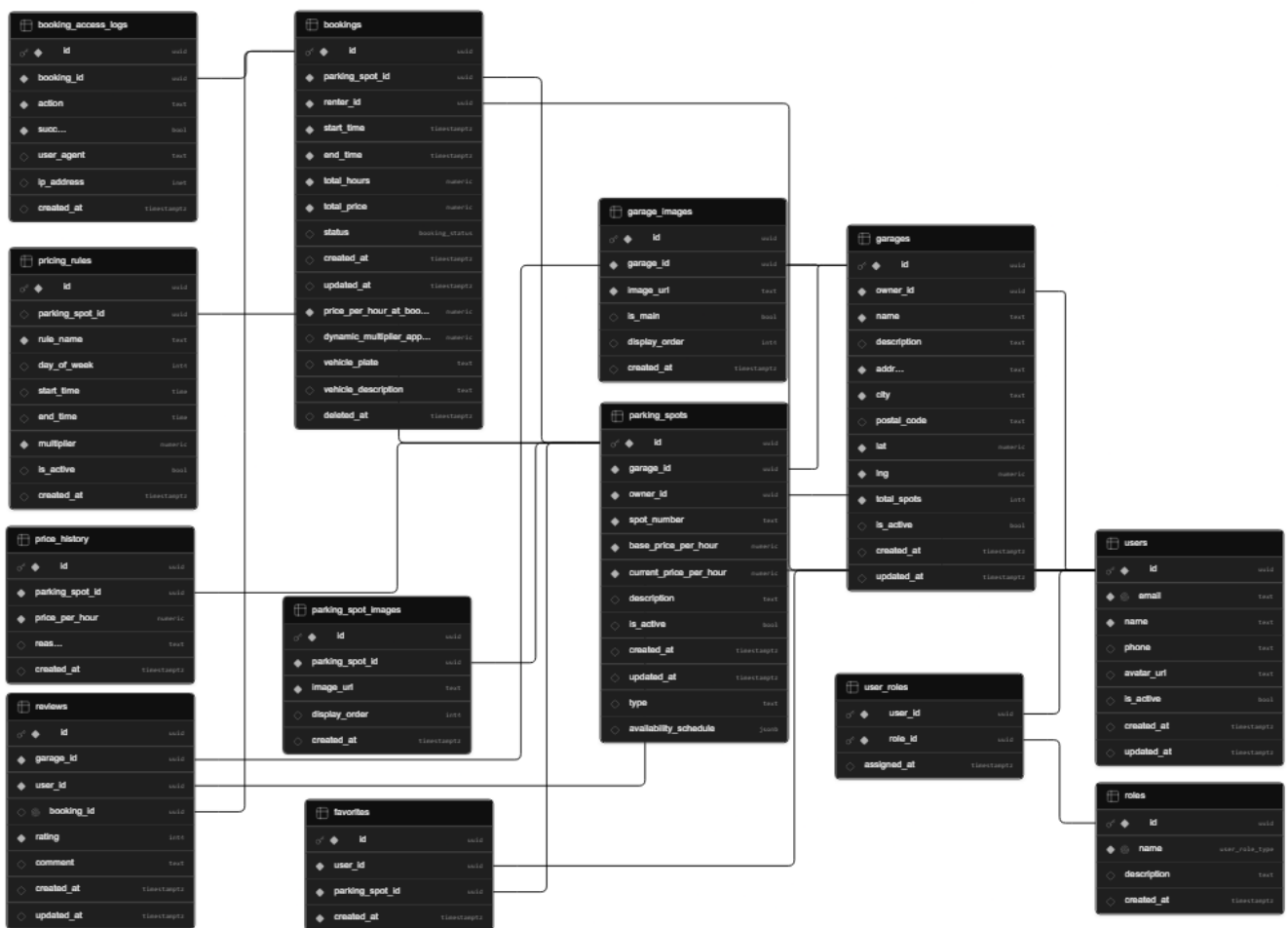


Figura 3.2: Diagrama entidad-relación del esquema de base de datos de Parky.

3.7.2. Políticas RLS

Las políticas de *Row Level Security* (RLS) de PostgreSQL filtran automáticamente las filas que cada usuario puede leer o modificar antes de devolver cualquier resultado, sin que el código del cliente intervenga. Esto significa que aunque un arrendatario manipulara la petición para incluir el identificador de otro usuario, la base de datos rechazaría la operación sin necesidad de ninguna validación adicional en el servidor de aplicación. Las cuatro políticas principales de Parky se recogen en la Tabla 3.10: cada una vincula el acceso a la identidad autenticada mediante `auth.uid()`, de modo que arrendadores y arrendatarios operan sobre subconjuntos de datos completamente aislados entre sí.

Tabla 3.10: Políticas de seguridad a nivel de fila (RLS) del esquema de Parky.

Tabla	Operación	Condición de acceso
profiles	SELECT, UPDATE	Solo el propio usuario puede leer o modificar su perfil (<code>auth.uid() = user_id</code>).
bookings	SELECT	Arrendador: ve reservas de sus plazas. Arrendatario: solo sus propias reservas (<code>tenant_id = auth.uid()</code>).
bookings	INSERT	Solo usuarios con rol arrendatario pueden crear reservas a su nombre.
reviews	INSERT	Solo si la reserva referenciada pertenece al usuario y está en estado completada.

3.7.3. Trigger de integridad de reservas

La tabla `bookings` está protegida por dos triggers que garantizan la integridad de las reservas a nivel de base de datos. El primero, `prevent_booking_overlap`, impide que se creen dos reservas solapadas en la misma plaza: antes de evaluar el solapamiento, el trigger adquiere un bloqueo explícito sobre la fila de `parking_spots` correspondiente mediante `SELECT ...FOR UPDATE`, serializando el acceso concurrente a la misma plaza y garantizando que dos transacciones simultáneas no pasan el chequeo de forma simultánea bajo el nivel de aislamiento `READ COMMITTED` que PostgreSQL usa por defecto. Adicionalmente, aplica un espaciado de 15 minutos entre reservas consecutivas (con el fin de garantizar el tiempo necesario a sacar el coche o en su defecto a la grúa). El segundo, `enforce_booking_status_machine`, restringe las transiciones del estado de la reserva a la secuencia pendiente → confirmada → activa → completada, impidiendo que una reserva cancelada pueda reactivarse.

PostgreSQL permite una alternativa para detectar solapamientos temporales mediante restricciones `EXCLUDE` con rangos de tipo `tstzrange` e índices GiST. Esta opción fue considerada y descartada por dos razones. La primera es que las restricciones `EXCLUDE` se limitan a comparar rangos y no admiten lógica adicional dentro de la misma expresión, de modo que incorporar el buffer de 15 minutos habría requerido una función de comparación personalizada, añadiendo complejidad sin beneficio claro. La segunda es que el trigger produce mensajes de error estructurados que la capa de aplicación puede interpretar directamente para mostrar al conductor el motivo del rechazo, mientras que una violación de restricción `EXCLUDE` devuelve un error genérico de integridad referencial.

Capítulo 4

Estudio de viabilidad económica

Este capítulo analiza la viabilidad económica de Parky en un escenario de despliegue profesional real y que haría falta para convertir el prototipo actual en un producto comercializable. Se detallan las brechas entre el prototipo y una versión lista para el mercado, el plan de desarrollo con su coste asociado, y la estimación del retorno de la inversión a medio plazo.

4.1. Del prototipo al producto comercial

El prototipo que este TFG documenta es totalmente funcional de extremo a extremo ya que un usuario puede registrarse, publicar una plaza, hacer una reserva, pagar con el motor de precios activo y abrir el acceso al garaje mediante SmartAccess pero lo que no cubre es todo lo que un producto final necesita antes de que alguien ponga dinero real en él.

La diferencia más importante es el procesamiento de pagos. El prototipo implementa la arquitectura de *Stripe Connect* (datos de la transacción, desglose de comisión e IGIC, modelo de pagos en base de datos) pero no ejecuta llamadas reales a la API. El porqué de esto no es un descuido, sino que *Stripe Connect* exige que la cuenta del propietario del *marketplace* esté verificada como empresa y que cada vendedor complete un proceso de *onboarding* con documentación de identidad que, en Canarias, implica acreditación fiscal ante la AEAT y alta en el Registro Mercantil, condiciones fuera del alcance de un proyecto académico. Para la versión comercial, esta integración es el primer requisito no técnico que bloquea el lanzamiento, y que se debe implementar para generar ingresos.

El segundo bloque pendiente es la presencia en tiendas de aplicaciones. El APK de Android funciona y se ha validado en dispositivos reales, pero publicarlo en Google Play exige superar una revisión de contenido y tener una cuenta de desarrollador activa. La versión iOS no existe porque Capacitor requiere un Mac con licencia de desarrollo para compilarla, y la revisión de App Store añade tiempos de respuesta de varios días además de una cuenta de desarrollador con un desembolso obligatorio de **100 euros**. Para el lanzamiento oficial, donde el uso móvil es predominante, ambas plataformas son obligatorias.

También quedan obligaciones legales. Al intermediar entre propietarios y conductores, la plataforma asume responsabilidad civil ante conflictos como daños en la plaza o disputas de facturación. Una versión comercial necesita condiciones generales de uso redactadas por un jurista, política de privacidad conforme a la ley vigente y un mecanismo de resolución de disputas. Finalmente, operar con usuarios reales exige monitorización más allá de las métricas básicas de Vercel y Supabase: alertas sobre errores en tiempo real, análisis de rendimiento por endpoint y un canal de soporte accesible.

Un coste que la versión comercial debe cuantificar, es el del equipamiento de acceso físico por plaza. Según lo descrito en la sección 5.6.1, la instalación de un módulo relé o un punto de acceso WiFi supone entre 30 y 70 € por plaza. Con las 990 plazas proyectadas para el mes 20, el coste agregado de hardware se situaría entre 30.000 y 70.000 €. Dependiendo de si ese desembolso lo asume el propietario o la plataforma, se traduce respectivamente en una barrera de captación en el lado de

la oferta que el modelo de ROI no recoge, o en una partida de CAPEX adicional no incluida en la Tabla 4.3. Para un despliegue piloto concentrado en comunidades con múltiples plazas, el coste por plaza es amortizable y puede negociarse con los administradores de fincas como condición del alta en la plataforma.

4.2. Funcionalidades de la versión avanzada

Más allá de completar los requisitos del lanzamiento, una versión avanzada incorporaría funcionalidades que ampliarían significativamente el atractivo de la plataforma.

Detección pasiva de estacionamiento. Capacitor puede monitorizar en segundo plano la conexión Bluetooth con el manos libres del vehículo. Al apagar el motor, el dispositivo registra la geolocalización y confirma la ocupación de la plaza sin intervención del usuario; al arrancar, la marca automáticamente como libre, maximizando el tiempo de uso disponible.

Pases temporales y abonos recurrentes. Con *Stripe Billing* se añadirían dos modalidades: un *EventPass* que agrupa plazas cercanas a recintos como el Estadio Heliodoro Rodríguez López a tarifa fija durante eventos, y un *SeasonPass* para conductores habituales con trayectos repetidos, como el personal de la Universidad de La Laguna. Ambos amplían el espectro de clientes y fidelizan usuarios recurrentes.

Validación automática ante la ZBE. Al confirmar una reserva dentro de la ZBE de Santa Cruz de Tenerife, un *webhook* de Supabase consulta la plataforma municipal y recibe un JWT firmado que exime al conductor de sanción administrativa, de forma completamente transparente para el usuario.

Tarificación de recarga para vehículos eléctricos. Para plazas con cargador doméstico, el motor de precios incorpora el coste eléctrico del PVPC¹ en tiempo real. Stripe Connect separa el coste energético del aparcamiento al liquidar el servicio, incentivando la oferta de plazas con punto de recarga.

Precios dinámicos por demanda contextual. El motor actual usa multiplicadores estáticos por día y tramo horario. Esta extensión añadiría tres señales: eventos próximos detectados vía APIs de agenda cultural y deportiva, ocupación histórica por franja horaria en la zona, y días de inactividad de la plaza, haciendo el sistema adaptativo a las condiciones reales del mercado.

¹**PVPC** (*Precio Voluntario para el Pequeño Consumidor*): Tarifa regulada de electricidad en España cuyo precio varía por hora según el mercado mayorista.

Programa de reputación y verificación de usuarios. Un sistema de puntuación bidireccional (inspirado en Airbnb y Uber) donde propietarios y conductores se califican mutuamente tras cada transacción. Los usuarios con reputación sostenida por encima de 4,5 puntos acceden a tarifas reducidas en un 5 % y a plazas exclusivas. La verificación mediante documento de identidad y datos bancarios genera un distintivo de *usuario verificado* que reduce fraudes y aumenta la confianza, especialmente en plazas de zonas premium.

Suscripciones y cancelación flexible. Los usuarios pueden configurar reservas recurrentes en franjas fijas con descuentos escalonados: 5 % por cinco reservas mensuales confirmadas, 10 % por diez. Una política de cancelación sin penalización con al menos 24 horas de antelación completa el modelo, generando ingresos predecibles sin comprometer la fluidez de la plataforma.

Área de búsqueda personalizada en el mapa. En lugar de un radio fijo, el usuario podría dibujar un polígono personalizado sobre el mapa para definir su zona de interés, tal como ya hacen plataformas como Idealista. La búsqueda se ajusta dinámicamente a esa área, ofreciendo una experiencia más flexible y adaptada a las necesidades reales de cada conductor.

4.3. Planificación y coste de la versión comercial

La planificación sigue un ciclo de vida en cinco fases, con pruebas en paralelo al desarrollo a partir del quinto mes. El equipo se contrata mediante subcontratación a una consultora de software, de modo que la tarifa horaria consolida ya los costes de seguridad social, equipamiento y margen de intermediación. La figura 4.2 muestra el diagrama y la figura 4.1 el coste del proyecto exportado desde ProjectLibre.

Análisis (35 días). Recogida de requisitos, modelado de la lógica de reservas y tarifas, revisión de la normativa ZBE y validación del tratamiento fiscal del IGIC. Participan el Jefe de Proyecto, el Analista y el Arquitecto de Software.

Diseño (30 días). Definición de la arquitectura: esquema PostgreSQL con RLS, selección del ecosistema tecnológico y elaboración de prototipos de interfaz para web y Android.

Desarrollo (142 días). Cuatro frentes en paralelo: *backend* con Supabase y Edge Functions (80 días), *frontend* web en React 18 (85 días, camino crítico), aplicación Android con Capacitor (61 días) y módulo SmartAccess (35 días). La integración completa de Stripe Connect se ejecuta dentro del bloque de *backend*.

Pruebas (91 días). Fase paralela desde el quinto mes: pruebas unitarias, de integración, de rendimiento bajo carga concurrente, de usabilidad con usuarios reales y de aceptación previas al despliegue.

Despliegue (12 días). Configuración de entornos de producción en Vercel y Supabase, publicación en Google Play y App Store, y auditoría de seguridad previa al lanzamiento.

		Costo	Nombre	Duración	Inicio	Terminado
1		245325,00 €	 Parky	310 days	1/1/26, 8:00	10/3/27, 17:00
2		18380,00 €	 Análisis	35 days	1/1/26, 8:00	18/2/26, 17:00
3		6960,00 €	Análisis de requisitos	12 days	1/1/26, 8:00	16/1/26, 17:00
4		4960,00 €	Análisis de funcionalidades	10 days	19/1/26, 8:00	30/1/26, 17:00
5		2600,00 €	Análisis de costes y viabilidad	5 days	2/2/26, 8:00	6/2/26, 17:00
6		2300,00 €	Elección y justificación de tecnologías	5 days	9/2/26, 8:00	13/2/26, 17:00
7		1560,00 €	Acta de constitución + reunión kickoff	3 days	16/2/26, 8:00	18/2/26, 17:00
8		19986,00 €	 Diseño	30 days	19/2/26, 8:00	1/4/26, 17:00
9		5500,00 €	Diseño de la arquitectura del sistema	10 days	19/2/26, 8:00	4/3/26, 17:00
10		3600,00 €	Diseño esquema BD + políticas RLS	8 days	19/2/26, 8:00	2/3/26, 17:00
11		2608,00 €	Diseño interfaces web (UX/UI)	8 days	5/3/26, 8:00	16/3/26, 17:00
12		1956,00 €	Diseño interfaces móvil	8 days	5/3/26, 8:00	16/3/26, 17:00
13		4032,00 €	Prototipado y revisión	7 days	17/3/26, 8:00	25/3/26, 17:00
14		2290,00 €	Revisión y validación del diseño	5 days	26/3/26, 8:00	1/4/26, 17:00
15		157072,00 €	 Desarrollo	142 days	2/4/26, 8:00	16/10/26, 17:00
16		5200,00 €	Configuración entorno	10 days	2/4/26, 8:00	15/4/26, 17:00
17		27860,00 €	Backend: Auth, RLS, DAL, APIs, Edge Functions	80 days	16/4/26, 8:00	5/8/26, 17:00
18		9500,00 €	Integración Stripe Connect + webhooks	25 days	14/5/26, 8:00	17/6/26, 17:00
19		46112,00 €	Frontend web: React + TS + Tailwind + rutas	88 days	16/4/26, 8:00	17/8/26, 17:00
20		24400,00 €	App móvil: Capacitor + Android + plugins nativos	61 days	16/4/26, 8:00	9/7/26, 17:00
21		9800,00 €	SmartAccess: IoT + QR + apertura remota	35 days	27/5/26, 7:00	14/7/26, 17:00
22		22000,00 €	Integración de todos los componentes	25 days	17/8/26, 7:00	18/9/26, 17:00
23		12200,00 €	Validaciones internas + correcciones	20 days	21/9/26, 8:00	16/10/26, 17:00
24		37352,00 €	 Pruebas	91 days	19/10/26, 8:00	22/2/27, 17:00
25		9280,00 €	Pruebas unitarias (backend + frontend + DAL)	20 days	19/10/26, 8:00	13/11/26, 17:00
26		10850,00 €	Pruebas de integración (API + Stripe + SmartAccess)	25 days	16/11/26, 8:00	18/12/26, 17:00
27		4620,00 €	Pruebas de rendimiento y carga	15 days	21/12/26, 8:00	8/1/27, 17:00
28		5342,00 €	Pruebas de usabilidad (web + Android)	15 days	12/1/27, 8:00	1/2/27, 17:00
29		7260,00 €	Pruebas de aceptación	15 days	2/2/27, 8:00	22/2/27, 17:00
30		4400,00 €	 Despliegue	12 days	23/2/27, 8:00	10/3/27, 17:00
31		2300,00 €	Configuración entorno de producción	5 days	23/2/27, 8:00	1/3/27, 17:00
32		1260,00 €	Despliegue web y backend	4 days	2/3/27, 8:00	5/3/27, 17:00
33		840,00 €	Publicación app Android en Play Store	3 days	8/3/27, 8:00	10/3/27, 17:00

Figura 4.1: Coste del proyecto Parky exportado desde ProjectLibre (enero 2026 – marzo 2027)

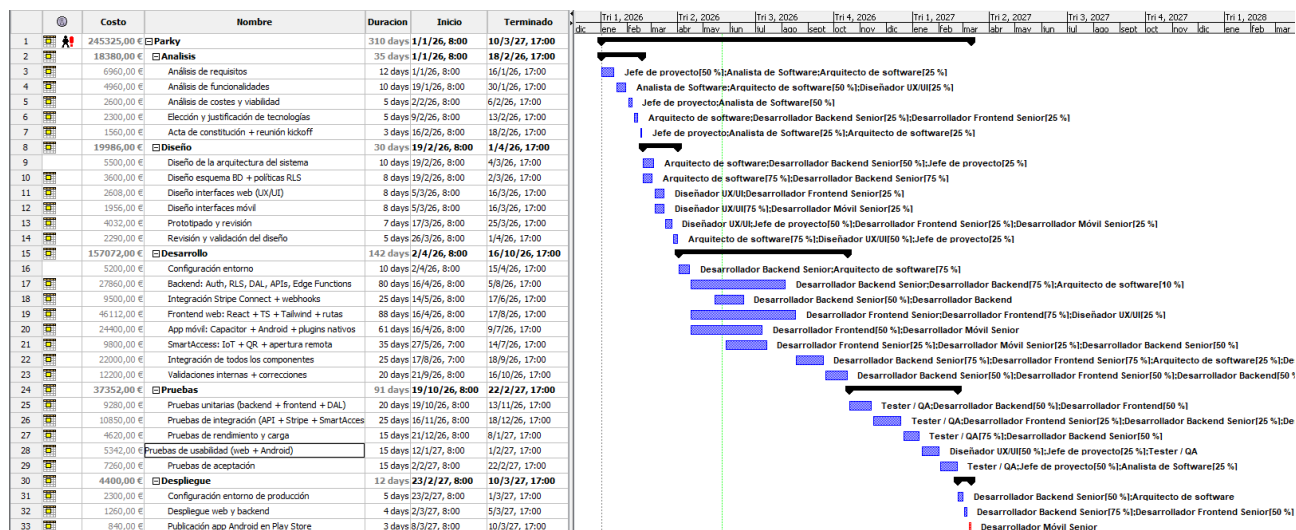


Figura 4.2: Diagrama de Gantt del proyecto Parky exportado desde ProjectLibre (enero 2026 – marzo 2027)

4.3.1. Equipo y coste del proyecto

El equipo mínimo necesario cubre diez perfiles técnicos. Al subcontratar a una consultora, la tarifa horaria absorbe los costes de seguridad social, licencias de software internas y equipamiento de la agencia. El proyecto requiere además hardware específico para validar la experiencia nativa en dispositivos móviles.

Perfil técnico	Horas est.	Tarifa (€/h)	Coste (€)
Jefe de Proyecto	300,00	45	13.500,00
Analista de Software	224,00	40	8.960,00
Arquitecto de Software	478,00	40	19.120,00
Diseñador UI/UX	444,00	32	14.208,00
Desarrollador Frontend Senior	1.106,00	35	38.710,00
Desarrollador Frontend	1.032,00	30	30.960,00
Desarrollador Móvil Senior	758,00	35	26.530,00
Desarrollador Backend Senior	1.242,00	35	43.470,00
Desarrollador Backend	790,00	30	23.700,00
Tester / QA	644,00	28	18.032,00
Subtotal personal	7.018,00	—	237.190,00

Tabla 4.1: Presupuesto por perfil técnico.

Concepto	Uds.	Coste (€)
Portátil de desarrollo	4	4.800,00
Smartphone iOS (pruebas nativas)	2	1.800,00
Smartphone Android (pruebas nativas)	2	1.200,00
Licencias IDE (JetBrains)	1	320,00
Dominio web y registro DNS	1	15,00
Subtotal material		8.135,00
Subtotal personal		237.190,00
Coste total del proyecto		245.325,00

Tabla 4.2: Recursos materiales y coste total del proyecto.

El coste de desarrollo calculado asciende a **245.325,00 €**. Sobre esta cifra se añade un colchón estructurado para absorber imprevistos técnicos, cubrir los gastos operativos durante el período de rodaje y financiar la campaña publicitaria de lanzamiento en Tenerife. La contingencia técnica se fija en el **10 %** del coste de desarrollo (24.533,00 €), porcentaje habitual en proyectos de software con integraciones externas como *Stripe Connect* o las revisiones de *App Store*. Los gastos legales y de *compliance* comentados anteriormente en la sección 4.1 se estiman en 4.500,00 €. Los primeros seis meses de operativa se presupuestan por adelantado: campaña de lanzamiento en Canarias mediante *Meta Ads* y *TikTok Ads* ($1.200,00 \text{ €} / \text{mes} \times 6 = 7.200,00 \text{ €}$), infraestructura *cloud* en tier de producción ($200,00 \text{ €} / \text{mes} \times 6 = 1.200,00 \text{ €}$) y gestión y contabilidad ($200,00 \text{ €} / \text{mes} \times 6 = 1.200,00 \text{ €}$). Una reserva de imprevistos operativos de aproximadamente 3.000,00 € completa el colchón. La **inversión total recomendada asciende a 287.000,00 €**, recogida en la tabla 4.3. Los seis meses de gastos operativos incluidos en la inversión inicial quedan reflejados mes a mes en la tabla 4.5 para mostrar el flujo de caja real del período de rodaje.

Concepto	Justificación	Importe (€)
Subtotal personal (10 perfiles, 310 días)	ProjectLibre	237.190,00
Hardware y licencias	Portátiles, iOS, Android, IDE, dominio	8.135,00
Subtotal desarrollo		245.325,00
Contingencia técnica (10 %)	Stripe, App Store, bugs post-lanzamiento	24.533,00
Legal y <i>compliance</i>	RGPD, condiciones de uso, AEAT Canarias	4.500,00
Campaña de lanzamiento (6 meses)	Meta Ads + TikTok Ads	7.200,00
Infraestructura <i>cloud</i> (6 meses)	Supabase Pro + Vercel Pro + Sentry	1.200,00
Gestión y contabilidad (6 meses)	Declaraciones IGIC + contabilidad mensual	1.200,00
Reserva de imprevistos operativos	Colchón de caja para ajustes de producto	3.042,00
Inversión total recomendada		287.000,00

Tabla 4.3: Desglose de la inversión inicial recomendada para Parky.

4.4. Modelo de comercialización

Parky opera como un *marketplace* bilateral en el que la plataforma intermediaria pone en contacto a propietarios de plazas de garaje con conductores, sin poseer el inventario físico. La estrategia de monetización se articula en tres vías independientes: una comisión transaccional por cada reserva completada, una suscripción mensual para propietarios con carteras de varias plazas, y micropagos de visibilidad premium en el mapa.

4.4.1. Comisión transaccional asimétrica

Cada reserva completada genera un ingreso para la plataforma mediante una estructura de comisión asimétrica. El precio base lo fija el propietario de la plaza ayudado de referencias de la app siendo el referente de mercado en Santa Cruz de Tenerife los parkings públicos del centro (1,50 €/h en el Intercambiador, tarifa plana de 9,95–10,95 €/día en Plaza de España [49]), por lo que una plaza privada a **0,90 €/h** ofrece un ahorro del 40 % y resulta competitiva. La reserva tipo más frecuente se entiende que sería la jornada laboral de ocho horas y esto supone un precio base de **7,20 €**.

El conductor abona sobre ese importe una **comisión de servicio del 15 %** más una tasa fija de gestión de 0,50 €, lo que eleva el precio final desembolsado a **8,78 €**. Por su parte, el propietario cede una **comisión del 8 %** sobre el precio base, notablemente inferior al 30 % que aplica JustPark en el mercado anglosajón [50], lo que facilita la captación en un mercado local sin competencia directa previa. Tras descontar el cargo de *Stripe Connect* para cuentas europeas (1,5 % más 0,25 € fijos, conforme a la tarifa publicada por Stripe para pagos en euros), el **ingreso neto por transacción para la plataforma se sitúa en 1,77 €**.²

4.4.2. Parky Pro: suscripción para arrendadores con cartera

Los propietarios con más de dos plazas o los administradores de fincas pueden contratar una suscripción mensual vía *Stripe Billing* que elimina la comisión del arrendador a cambio de una cuota fija recurrente y hay dos niveles:

Basic Pro (14,99 €/mes). Acceso a un panel de analítica básica con datos de ocupación, ingresos históricos y rendimiento por plaza; actualización masiva de tarifas para hasta cinco plazas; prioridad en el canal de soporte.

Advanced Pro (29,99 €/mes). Exención completa de la comisión de arrendador, gestión de hasta 25 plazas, integración con el calendario de eventos de Tenerife (partidos en el Estadio Heliodoro Rodríguez López, congresos en la Universidad de La Laguna, carnaval de Santa Cruz) y soporte para la declaración trimestral del IGIC.

Dado que los administradores de fincas contactados durante la fase de captación de inventario son el *target* natural de este plan, se estima un suscriptor Pro por cada diez plazas activas en la plataforma. Asumiendo un reparto del 60 % de suscriptores en Basic Pro y un 40 % en Advanced Pro, el precio

²Cálculo exacto: ingresos brutos de plataforma $(1,58+0,58) = 2,16$ € menos cargo Stripe $(1,5 \% \times 8,78 + 0,25) = 0,38$ €, resultado 1,78 €; se usa 1,77 € aplicando la política de redondeo a dos decimales sobre todos los cálculos intermedios.

medio ponderado se sitúa en **20,99 € por suscriptor al mes**. Ambos niveles generan ingresos predecibles para la plataforma incluso en períodos de baja demanda transaccional.

4.4.3. Visibilidad premium

Los propietarios pueden adquirir mejoras de posicionamiento mediante un pago único *in-app*:

- **Destacado de Zona (1,99 € / 3 días)**: eleva algorítmicamente la plaza en los listados de búsqueda cercanos y diferencia el marcador en el mapa de *Leaflet* mediante un color exclusivo.
- **Posicionamiento Urgente (4,99 € / 7 días)**: muestra la plaza con prioridad a los conductores que se aproximan a la zona durante eventos de alta demanda, como partidos en el Estadio Heliodoro o congresos en la Universidad de La Laguna.

Asumiendo un 60 % de compras en Destacado de Zona y un 40 % en Posicionamiento Urgente, el precio medio ponderado por destacado es de **3,19 €**. Se estima que el 10 % de las plazas activas contrata al menos un destacado al mes, lo que proporciona ingresos directos sin incrementar la comisión base ni penalizar a los propietarios no suscritos.

4.5. Estimación del retorno de la inversión

El *retorno de la inversión (ROI)* confronta los gastos acumulados (los cuales parten de la inversión inicial de 287.000,00 € y crecen con los operativos mensuales) con los ingresos generados por las tres vías de monetización descritas en la sección anterior 4.4.

4.5.1. Hipótesis de partida

El modelo proyecta la evolución de Parky a partir de cuatro hipótesis cuantificables y ancladas al contexto del área metropolitana de Tenerife y la tabla 4.4 recoge los parámetros clave del modelo de proyección financiera de Parky. Estas **hipótesis** se basan en datos de mercado, referencias de plataformas similares y las características específicas del entorno local.

Inventario inicial. La campaña de captación pre-lanzamiento, dirigida a administradores de fincas y presidentes de comunidades de propietarios, permitiría alcanzar **100 plazas activas en el primer mes comercial**. Con 3-4 comunidades grandes captadas (cada una con entre 20 y 35 plazas) y algunos propietarios individuales se cubre este objetivo. El crecimiento posterior es orgánico: cada propietario satisfecho refiere de media a 0,5 nuevos propietarios al mes; esta hipótesis se calibra como estimación conservadora, coherente con el dinamismo de crecimiento observado en plataformas de economía colaborativa [3], sin que existan datos locales directamente comparables para este mercado. La curva alcanza las **990 plazas en el mes 20** y continúa creciendo de forma moderada (1.020 en el mes 22, 1.050 en el mes 24) a medida que el referido orgánico mantiene un ritmo bajo pero sostenido más allá del horizonte de proyección.

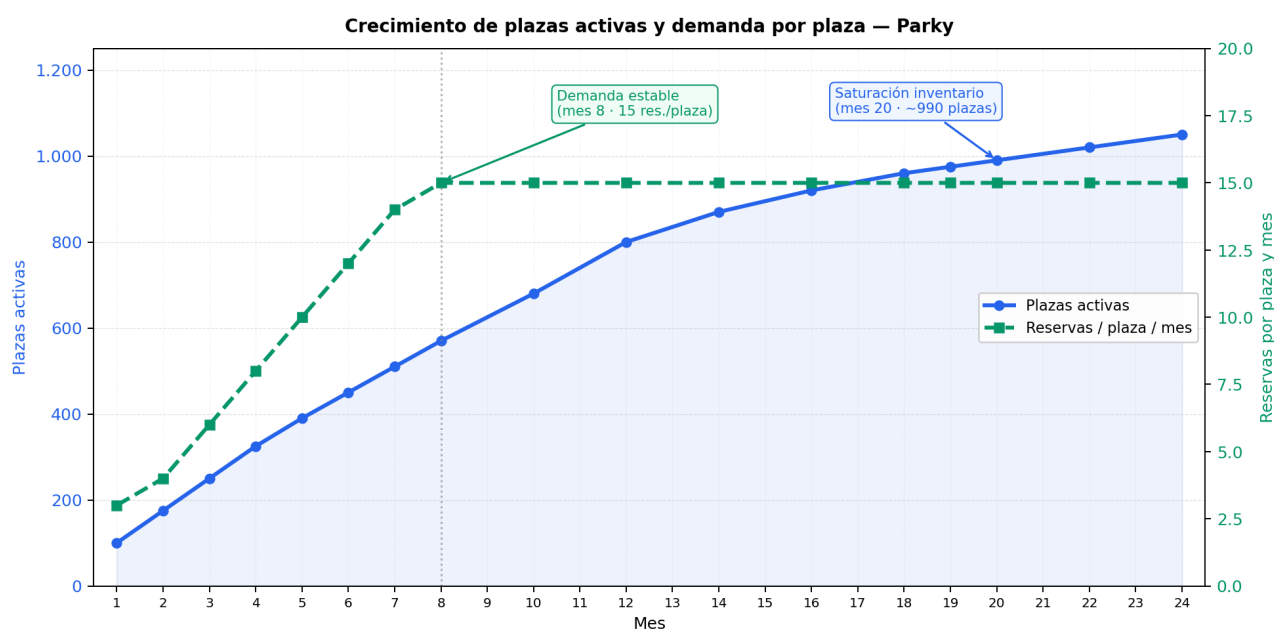


Figura 4.3: Evolución del inventario de plazas activas y la demanda por plaza a lo largo de los veinticuatro meses proyectados.

Demanda por plaza. La plataforma arranca con **3 reservas por plaza al mes** (equivalente a alquilar aproximadamente un día de cada diez la plaza) y puede escalar progresivamente hasta estabilizarse en **15 reservas por plaza al mes a partir del mes 8**, cifra conservadora si se compara con los parkings de rotación del centro de Santa Cruz de Tenerife, que operan con tasas de ocupación superiores al 60 % en jornada laboral [49]. La hipótesis asume que una plaza privada se alquila aproximadamente la mitad de los días laborables del mes durante una jornada de trabajo media, por lo que la demanda potencial, con precios un 40 % inferiores a los parkings comerciales, es suficiente para sostener ese ritmo.

Ingreso por plaza con reservas estables. Con 15 reservas mensuales, las tres vías de monetización generan un ingreso medio de **28,97 € por plaza al mes**: 26,55 € por comisiones netas de transacción, 2,10 € por la parte proporcional de Parky Pro y 0,32 € por visibilidad premium (todo esto es meramente estimado y puede variar). Los importes de la tabla se presentan redondeados a la unidad más próxima; los cálculos intermedios se realizan con cuatro decimales, por lo que pueden existir diferencias inferiores a un euro respecto a la aplicación directa de los porcentajes sobre las cifras redondeadas.

Gastos operativos. Los costes post-lanzamiento se fijan en **1.830,00 €/mes** durante los seis primeros meses (infraestructura, marketing intensivo, gestión y soporte) y se reducen a **1.130,00 €/mes** a partir del séptimo mes, cuando la campaña de lanzamiento en Meta Ads y TikTok Ads (que representaba la mayor parte del gasto variable de los primeros seis meses) finaliza. A partir del mes 7, la adquisición pasa a apoyarse en referencias entre propietarios y en posicionamiento orgánico, cuyo coste marginal es sustancialmente inferior al de la publicidad pagada. La infraestructura *cloud* y el soporte escalan de forma sublineal con el número de usuarios a este nivel de tráfico, por lo que no generan un incremento significativo del gasto al crecer la base de plazas. Los 1.830,00 €/mes de la etapa de captación se desglosan en campaña publicitaria activa en redes (700,00 €/mes; los

3.000,00 € restantes del presupuesto de campaña recogido en la Tabla 4.3 corresponden a producción creativa y configuración inicial, ambas costes únicos del mes 1), infraestructura *cloud* en tier de producción como son Supabase Pro, Vercel Pro y Sentry (380,00 €/mes, con la diferencia respecto al importe estimado cubierta por la reserva de imprevistos), gestión y contabilidad (200,00 €/mes) y soporte técnico y atención al usuario (550,00 €/mes). Al término del mes 6, el cese del gasto publicitario activo elimina los 700,00 €/mes correspondientes, reduciendo el operativo estable a $380 + 200 + 550 = 1.130,00$ €/mes, sin que el resto de partidas varíe.

Parámetro	Valor inicial	Valor estable
Plazas activas	100 (mes 1)	≈990 (mes 20)
Reservas por plaza y mes	3 (mes 1)	15 (desde mes 8)
Ingreso medio por plaza y mes	—	28,97 € (desde mes 8)
Gastos operativos mensuales	1.830,00 €	1.130,00 € (desde mes 7)

Tabla 4.4: Parámetros clave del modelo de proyección financiera de Parky.

4.5.2. Proyección y punto de equilibrio

La tabla 4.5 recoge la evolución mensual de ingresos y gastos desde el primer mes comercial hasta el mes veinticuatro. Dado que los ingresos de Parky dependen directamente del volumen de reservas completadas, los primeros meses registran flujo de caja negativo mientras la base de conductores recurrentes crece hasta sostener una demanda estable por plaza.

El modelo alcanza el **equilibrio operativo en el mes 3**, cuando los ingresos mensuales superan por primera vez los gastos del período. Sin embargo, la recuperación íntegra de la inversión inicial de 287.000,00 € no se produce hasta el **mes 19**, cuando el balance acumulado se vuelve positivo por primera vez con un saldo de **14.951,00 €**. Este hito coincide con la consolidación de la base de suscriptores Parky Pro y con la demanda estabilizada en 15 reservas por plaza y mes. Al cierre del mes 24 el balance acumulado favorable asciende a **157.038,00 €**, confirmando la viabilidad del modelo a medio plazo bajo un escenario de crecimiento moderado y sin expansión fuera de Tenerife. Esto equivale a un ROI del **54,7 %** sobre la inversión inicial en un horizonte de dos años, calculado como $\text{ROI} = \text{beneficio neto acumulado} / \text{inversión inicial} = 157.038 / 287.000$.

La figura 4.5 ilustra la evolución del balance a lo largo de los veinticuatro meses.

Mes	Plazas	Res./plaza	Ing. mes (€)	Gastos mes (€)	Flujo neto (€)	Ing. acum. (€)	Balance (€)
1	100	3	773	1.830	-1.057	773	-288.057
2	175	4	1.662	1.830	-168	2.435	-288.225
3	250	6	3.260	1.830	+1.430	5.694	-286.796
4	325	8	5.388	1.830	+3.558	11.082	-283.238
5	390	10	7.846	1.830	+6.016	18.928	-277.222
6	450	12	10.646	1.830	+8.816	29.574	-268.406
7	510	14	13.871	1.130	+12.741	43.445	-255.665
8	570	15	16.512	1.130	+15.382	59.957	-240.283
10	680	15	19.698	1.130	+18.568	97.760	-204.740
12	800	15	23.174	1.130	+22.044	142.371	-162.389
14	870	15	25.202	1.130	+24.072	191.762	-115.258
16	920	15	26.651	1.130	+25.521	244.338	-64.942
18	960	15	27.809	1.130	+26.679	299.378	-12.162
19	975	15	28.244	1.130	+27.114	327.621	+14.951
20	990	15	28.678	1.130	+27.548	356.300	+42.500
22	1.020	15	29.547	1.130	+28.417	414.960	+98.900
24	1.050	15	30.416	1.130	+29.286	475.358	+157.038

Tabla 4.5: Evolución proyectada de ingresos y gastos acumulados. La fila sombreada en gris (mes 3) es el primer mes con flujo neto positivo; la fila en verde (mes 19) marca la recuperación íntegra de la inversión inicial de 287.000,00 €.

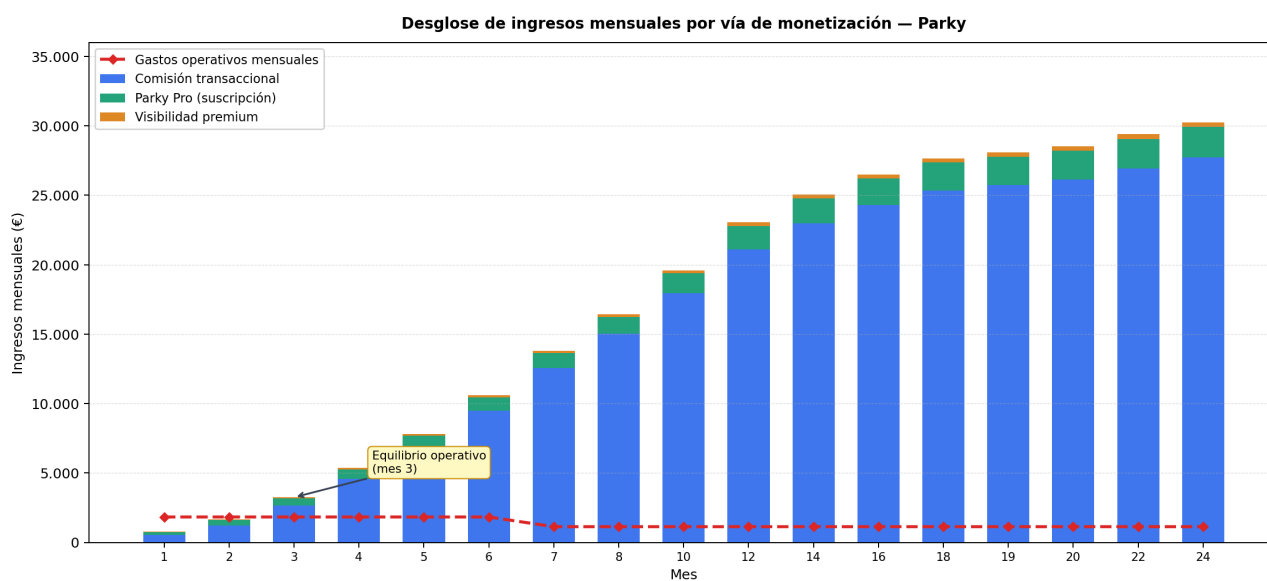


Figura 4.4: Desglose mensual de ingresos por vía de monetización. La línea discontinua roja representa los gastos operativos mensuales.

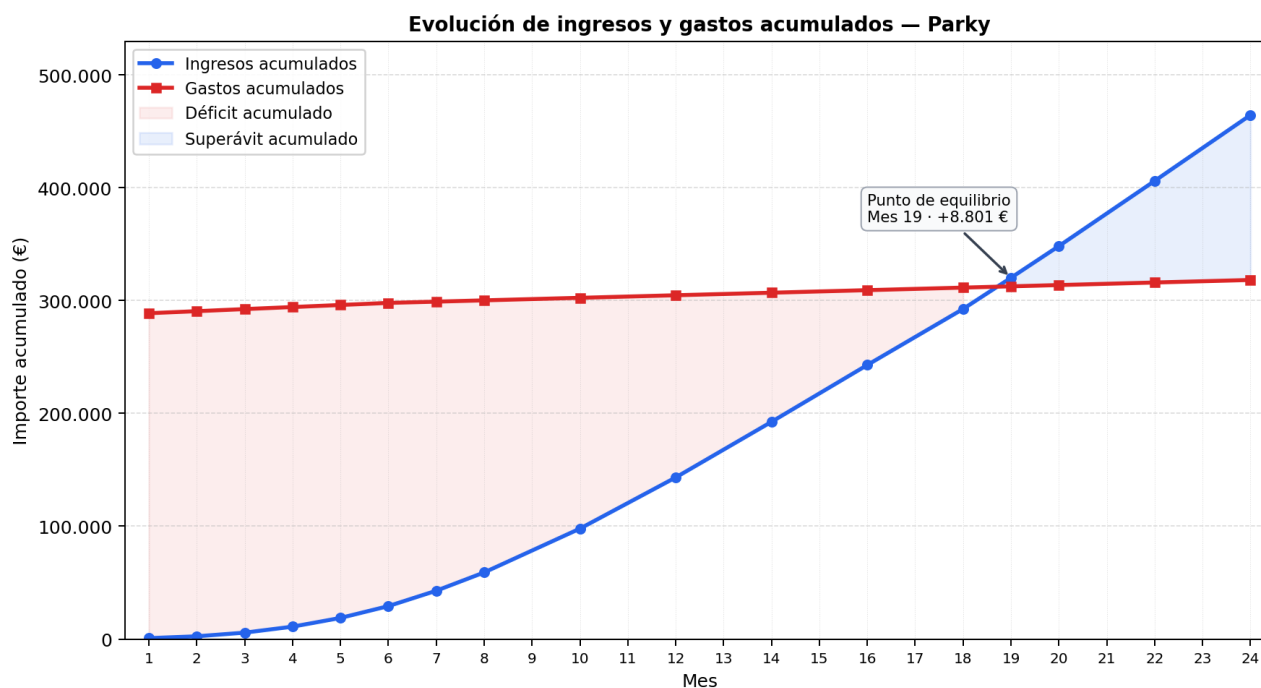


Figura 4.5: Evolución del balance acumulado a lo largo de los veinticuatro meses.

4.5.3. Análisis de sensibilidad

El escenario base del modelo principal alcanza el equilibrio en el mes 19 partiendo de 100 plazas iniciales y creciendo hasta las 990 plazas con la rampa descrita en la sección 4.5.1. Para evaluar la sensibilidad del modelo a las dos hipótesis más inciertas (el inventario y la demanda por plaza) sin depender de esa curva de captación, la tabla 4.6 presenta cuatro escenarios en **régimen estacionario**: cada escenario asume que la cartera de plazas y la demanda por plaza se alcanzan desde el primer mes. El *payback* resultante es por tanto inferior al del modelo principal (mes 19) y representa el tiempo mínimo de recuperación una vez alcanzada la operación plena, no el tiempo real desde el lanzamiento. Las variables unitarias son las mismas en todos los escenarios (1,77 € neto por reserva y 2,42 € mensuales por suscripción y visibilidad por plaza) y los costes operativos estabilizados de 1.130,00 €/mes.

Tabla 4.6: Análisis de sensibilidad del ROI bajo cuatro escenarios de operación.

Escenario	Plazas	Res./plaza/mes	Ingresos/mes	Payback
Conservador	500	8	8.290,00 €	40,1 meses
Base (demanda menor)	990	10	19.919,00 €	15,3 meses
Base actual	990	15	28.680,00 €	10,4 meses
Expansión insular	1.500	15	43.455,00 €	6,8 meses

El escenario conservador asume que la captación se limita al área central de Santa Cruz y que la frecuencia de uso por plaza no supera la mitad de la hipótesis base. Aun así, el payback no supera

los cuatro años y el modelo resulta viable. El escenario de expansión insular incorpora plazas en Las Palmas de Gran Canaria y otras capitales del archipiélago, donde la infraestructura técnica es directamente aplicable sin modificaciones. En todos los casos la variable más sensible es la cartera de plazas activas, ya que duplicar el número de plazas con la misma frecuencia de uso impacta más en el payback que duplicar la frecuencia manteniendo la misma cartera.

Capítulo 5

Implementación

Este capítulo describe las decisiones de implementación más relevantes de Parky, organizadas por módulo funcional.

5.1. Arquitectura por capas

El código fuente de Parky se organiza en módulos autónomos bajo `src/features/`, cada uno articulado en cuatro capas con responsabilidades bien diferenciadas. La capa de acceso a datos concentra todas las consultas a la base de datos y siempre especifica explícitamente las columnas necesarias, lo que evita que un cambio de esquema exponga campos sensibles de forma accidental. Por otro lado, la capa de servicio orquesta la lógica de negocio que cruza más de una entidad. Los hooks de React envuelven los servicios con gestión de caché y estados de carga, y los componentes son exclusivamente de presentación, sin acceso directo a la base de datos.

5.2. Módulo de autenticación

La autenticación se apoya en Supabase Auth, que emite tokens de sesión con refresco automático y admite tanto el registro con correo y contraseña como la entrada mediante Google, de forma que en ambos casos el sistema crea automáticamente el perfil del usuario en la base de datos sin necesidad de ninguna llamada adicional desde el cliente.

El momento de mayor relevancia desde el punto de vista del diseño es el primer inicio de sesión, cuando la aplicación detecta que el usuario todavía no tiene un rol asignado y al principio del diseño lo redirigía a la pantalla de selección para que elija entre Arrendador y Arrendatario, sin embargo, ahora cada vez que entra tiene el rol automático de Arrendatario y solo se cambia si crea un garaje. Una vez establecido, el rol queda fijo y las políticas de seguridad de la base de datos impiden que un usuario lo modifique desde el cliente, lo que garantiza que ningún arrendatario pueda acceder a las funciones de gestión de plazas ni viceversa. La protección de rutas se gestiona mediante un componente dedicado que comprueba en tiempo de ejecución si el usuario autenticado tiene permiso para acceder a la sección solicitada, redirigiendo al inicio de la aplicación en caso contrario.

5.3. Módulo de plazas y mapa

El inventario de aparcamiento se estructura en dos niveles jerárquicos, de modo que cada garaje agrupa la dirección, el propietario y las fotos generales del acceso, mientras que cada plaza individual registra su precio por hora, su disponibilidad horaria y sus propias imágenes. Separar ambas entidades permite que un mismo propietario gestione varias plazas bajo un único garaje sin duplicar la información de

ubicación, y hace que el mapa muestre un solo marcador por garaje incluso cuando dispone de varias plazas, lo que mejora la legibilidad cuando la densidad de puntos es alta.

El mapa se construye sobre Leaflet y recalcula los garajes visibles cada vez que el usuario mueve la vista o cambia el nivel de zoom, consultando únicamente los que tienen al menos una plaza activa dentro del área visible en ese momento. Los filtros disponibles, que permiten acotar por precio máximo, rango horario y tipo de plaza, se aplican en el servidor para minimizar la cantidad de datos transferidos, de forma que al cliente solo llegan los garajes que cumplen los criterios seleccionados.

Para convertir las direcciones escritas por los arrendadores en coordenadas geográficas, la aplicación utiliza la API de OpenCage a través de una función de servidor, que además verifica que las coordenadas obtenidas correspondan al territorio de Tenerife antes de persistirlas, con el propósito de impedir el registro de plazas fuera del ámbito del servicio. Esto significa que si existe una calle con el mismo nombre en otro lugar, no se registrarán plazas en esa ubicación sino que lo haga en Tenerife.

5.4. Módulo de reservas

El ciclo de una reserva comienza con el cálculo del precio estimado, que la aplicación muestra al usuario antes de confirmar la operación. El precio final, sin embargo, siempre lo determina el servidor a partir de las reglas de tarificación activas para esa plaza, que pueden incluir multiplicadores según el día de la semana o el tramo horario, de modo que el importe que se persiste en la base de datos no puede ser manipulado desde el cliente.

La integridad temporal del inventario se protege en dos niveles. La consulta de disponibilidad aplica un margen de **quince minutos antes y después de cada reserva existente**, de forma que dos vehículos nunca se crucen en el acceso al garaje al coincidir el final de una reserva con el inicio de la siguiente. El segundo nivel, y el técnicamente más relevante, es que el sistema rechaza los solapamientos dentro de la propia transacción de base de datos mediante un trigger de PostgreSQL, de modo que si dos usuarios intentan reservar la misma plaza al mismo tiempo solo una de las dos solicitudes tiene éxito, independientemente del número de peticiones simultáneas, algo que no sería posible garantizar si la comprobación se realizase en la capa de aplicación. La serialización se consigue mediante un bloqueo explícito (`SELECT ... FOR UPDATE`) sobre la fila de la plaza antes de la comprobación de solapamiento, lo que impide que dos transacciones concurrentes pasen el chequeo simultáneamente bajo el nivel de aislamiento `READ COMMITTED` que PostgreSQL usa por defecto.

El ciclo de vida de cada reserva sigue una transición de estados que solo admite transiciones válidas, pasando de pendiente a confirmada, de confirmada a activa y de activa a completada, mientras que la cancelación es posible desde cualquier estado no terminal. Las reservas nunca se eliminan físicamente de la base de datos sino que se marcan mediante una fecha de cancelación, lo que preserva el historial completo de transacciones y facilita las declaraciones fiscales trimestrales en caso de tener el plan acorde para realizar esto como se ve en la sección 4.4.2.

5.5. Módulo de pagos

El modelo de pagos se construye hipotéticamente sobre Stripe Connect, lo que significa que cada arrendador vincularía su cuenta bancaria a la plataforma antes de poder recibir ingresos. Cuando un arrendatario confirma una reserva, la plataforma crea una operación de pago en la que Stripe retiene automáticamente la comisión correspondiente y transfiere el saldo neto directamente a la cuenta del arrendador, sin que el dinero pase en ningún momento por una cuenta intermedia de la plataforma.

El aspecto más delicado del módulo es el tratamiento del IGIC. Este cálculo se realiza en el servidor, nunca en el cliente, tanto para evitar que un usuario manipule el importe fiscal declarado como para centralizar la lógica tributaria en un único punto que sea fácil de auditar. El desglose del precio, con el importe base, la comisión y el IGIC como valores independientes, se almacena en columnas separadas de la tabla de reservas para facilitar los informes trimestrales. Si el arrendatario cancela antes del inicio de la reserva o el pago no llega a completarse, la operación se invierte íntegramente y se emite el reembolso completo sin retención.

5.6. Módulo SmartAccess

SmartAccess es el módulo de control de acceso que integra las capas de autenticación, reservas y base de datos, y es el que permite al arrendatario abrir y cerrar el garaje desde la aplicación durante la ventana temporal de su reserva. Antes de registrar cualquier evento de acceso, el sistema verifica simultáneamente que el usuario autenticado es el titular de la reserva, que el estado de esta es *activo* y que el instante en el que se solicita el acceso cae dentro del intervalo reservado. Si alguna de las tres condiciones no se cumple, el servidor responde con un error genérico que no revela si la reserva existe pero pertenece a otro usuario, lo que impide que un atacante pueda identificar reservas ajenas simplemente observando las respuestas de la API (esto fue una de las pruebas hechas como atacante para verificar la fiabilidad de la implementación). Cada intento de apertura o cierre, con independencia de si tiene éxito, queda registrado con marca de tiempo y tipo de evento en la tabla `access_log`.

5.6.1. Modelo de acceso físico

La capa de software de SmartAccess aunque esté completa y actualmente genera un token de un solo uso vinculado al estado activo de la reserva, lo invalida al terminar el intervalo y registra cada intento en `access_log`, no tiene resuelto la pieza del medio, es decir, lo qué hay físicamente en la puerta del garaje que recibe esa señal y acciona el mecanismo de apertura.

En la práctica, los garajes residenciales del área metropolitana de Tenerife emplean principalmente portones con motor eléctrico y, en menor medida, puertas peatonales con cerradura electroimantada. Los dos admiten automatización con hardware disponible en el mercado, aunque con grados de complejidad distintos.

La integración más directa pasa por un módulo relé conectado a la central del portón. Un microcontrolador ESP32 o una Raspberry Pi Zero escucha un endpoint de la API y cierra el circuito del relé al recibir un token válido, lo que equivale a pulsar el botón del mando de apertura. El coste de hardware

ronda los 30–50 € por plaza y la instalación no requiere modificar el motor existente. Esta es la opción más viable para un piloto inicial donde haya bastantes plazas dentro de un mismo parking.

La alternativa inalámbrica recurre a Bluetooth Low Energy (BLE) como funcionan la mayoría de los dispositivos modernos de cerraduras inteligentes. La aplicación abre directamente una conexión con un módulo en la puerta sin pasar por internet, lo que elimina la dependencia de conectividad en el garaje. El inconveniente es el rango, ya que en garajes subterráneos con señal débil la conexión puede ser inestable a más de diez metros del módulo.

Existe una tercera opción que evita el hardware personalizado, que consiste en integrarse con sistemas de cerradura inteligente ya instalados previamente en comunidades de propietarios. Si el edificio ya tiene control de acceso centralizado para el portal, la integración con Parky se reduce a una llamada a la API del fabricante.

Desde la perspectiva del conductor, el flujo es idéntico en los tres casos. Pulsa «Abrir» durante su reserva activa, el servidor valida el token y envía la señal al actuador. La diferencia está en el actuador, no en la aplicación.

El código QR generado por RF-15 actúa como mecanismo de acceso complementario orientado a instalaciones con lector integrado en la barrera, equipamiento presente en aparcamientos comerciales o comunitarios modernos. En garajes residenciales sin ese hardware, el flujo de apertura se realiza exclusivamente mediante el botón en la aplicación, que envía la señal al actuador a través del servidor y el QR queda disponible como respaldo visual para el escenario de acceso delegado o para futuras integraciones con hardware comercial.

5.6.2. Gestión de incidencias y casos límite

Cualquier sistema de acceso automatizado tiene que dar respuesta a los escenarios en los que algo no sale según lo previsto. Los más probables en Parky son cinco.

El coche sigue dentro al finalizar la reserva. Cuando el intervalo de una reserva termina, la puerta no se cierra de forma automática aunque el conductor siga dentro, puesto que cerrar una barrera con un vehículo en el paso sería un riesgo físico inadmisibles. Lo que ocurre es que el token expira, la reserva transiciona a completada y el sistema envía una notificación al conductor indicando que su tiempo ha terminado. El propietario puede abrir la puerta desde su panel para recuperar el acceso. Una extensión lógica para la versión comercial sería aplicar un recargo por minuto adicional desde la hora de finalización, siguiendo el modelo habitual en parkings de rotación.

Sin conexión a internet en el garaje. Los garajes subterráneos suelen tener cobertura débil, de modo que si el servidor no puede comunicarse con el actuador, la apertura falla. Una primera solución pasa por resolver la apertura por BLE directamente desde el dispositivo del conductor, sin depender del servidor en ese momento. Una segunda alternativa, más sencilla de desplegar, consiste en instalar un punto de acceso WiFi en el garaje por un coste de 20–40 €. Para un piloto inicial, la opción más pragmática es exigir conectividad al garaje como requisito previo al registro en Parky; el modo offline por BLE quedaría como mejora para versiones posteriores y después de ser estudiado sus problemas y costes.

Móvil sin batería o QR inaccesible. Si el conductor llega al garaje con el móvil apagado, no hay acceso automático posible. La salida práctica sería proporcionar un código de respaldo de 7 dígitos enviado por SMS al confirmar la reserva. Este código funcionaría sin conexión en el momento del acceso, actuando como segundo factor offline.

Protección ante replay de tokens. Cada token se invalida en el servidor en cuanto se consume, sin esperar a que expire por tiempo. Para volver a entrar dentro del mismo intervalo, la aplicación solicita un token nuevo. Esto impide que un atacante capture el QR de otro conductor y lo use más tarde, dado que el token ya estará invalidado. El campo `nonce` en `access_log` registra el identificador único de cada intento, lo que permite detectar reutilizaciones incluso si proceden de redes distintas.

Reserva cancelada mientras el conductor está dentro. Si el propietario intentara cancelar una reserva con el conductor ya en el garaje, el token activo no se invalida de forma inmediata, ya que forzar esa invalidación podría dejar el vehículo atrapado sin salida. Por eso el control de estados de las reservas no permite cancelar reservas en estado activo desde el panel del propietario. El propietario puede registrar una incidencia, pero la reserva completa su ciclo normal antes de que se puedan tomar otras acciones.

5.7. Despliegue multiplataforma

Web. Vite compila la aplicación en modo producción generando un árbol de artefactos estáticos optimizados. Vercel recoge el repositorio de GitHub, ejecuta `npm run build` en cada *push* a `main` y distribuye el resultado en su CDN global. El archivo `vercel.json` redirige todas las rutas al `index.html` para que React Router DOM gestione la navegación sin recargas de página.

Android. Capacitor encapsula la aplicación web dentro de una *WebView* nativa de Android. El módulo `src/mobile/platform.ts` detecta en tiempo de ejecución si la aplicación corre en un dispositivo nativo o en un navegador, y activa o desactiva los plugins de Capacitor (haptics, barra de estado, almacenamiento nativo) de forma condicional. El APK se genera desde Android Studio sin modificar ningún componente de React.

5.8. Pruebas realizadas

Las pruebas del prototipo se organizaron en torno a los tres problemas de riesgo más relevantes del sistema: la integridad temporal del inventario de plazas, la seguridad del control de acceso y la corrección del modelo de pagos.

Solapamiento concurrente de reservas. Para verificar que el trigger de PostgreSQL descrito en la sección 5.4 funciona bajo carga simultánea, se lanzaron dos peticiones de reserva para la misma plaza y el mismo intervalo de forma concurrente desde dos sesiones de navegador independientes. En todos los casos exactamente una de las dos peticiones tuvo éxito y la otra recibió el mensaje de error estructurado devuelto por el trigger, confirmando que la protección opera a nivel de transacción y no depende de la velocidad de red ni del orden de llegada al servidor de aplicación.

Transición inválida de estado. El control de estados de las reservas se validó intentando forzar transiciones no previstas directamente contra la API como pasar de pendiente a completada sin pasar por activa, o cancelar una reserva ya en estado completada. El servidor rechazó todas las peticiones con el código de error correspondiente, sin producir estados inconsistentes en la base de datos.

Manipulación del tenant_id. Las políticas RLS del módulo de reservas garantizan que un usuario autenticado no puede leer ni modificar reservas ajenas. Se verificó enviando peticiones modificadas en las que el campo tenant_id contenía el identificador de otro usuario registrado. En todos los casos la política devolvió un resultado vacío o un error de autorización sin revelar si el recurso existía.

Seguridad de SmartAccess. Como se describe en la sección 5.6, se probó el endpoint de apertura de puerta con tokens pertenecientes a reservas de otro usuario y con tokens de reservas caducadas. El servidor devolvió un error genérico en ambos casos sin indicar si la reserva consultada existía, impidiendo que un atacante pudiera enumerar reservas ajenas observando las respuestas de la API.

Manipulación del precio desde el cliente. El motor de precios recalcula el importe final siempre en el servidor a partir de las reglas activas para cada plaza. Se verificó modificando el valor de precio enviado en el cuerpo de la petición de reserva. El sistema ignoró el valor del cliente y persistió el precio calculado en el servidor, de forma que la reserva se registró con el importe correcto independientemente del valor enviado.

Capítulo 6

Conclusiones y líneas futuras

El objetivo de este trabajo era demostrar que el problema de aparcamiento en el área metropolitana de Tenerife tiene solución técnica y económicamente viable, y los siete objetivos específicos definidos en la sección 1.5 quedan cubiertos por el prototipo. El resultado es un sistema que funciona de extremo a extremo, en el que un usuario puede registrarse, publicar una plaza, realizar una reserva, pagar y abrir el garaje sin salir de la aplicación.

La decisión técnica que más marcó el desarrollo fue delegar en PostgreSQL lo que habitualmente se gestiona en la capa de aplicación. Resolver los solapamientos de reservas con un trigger en la base de datos eliminó toda una clase de condiciones de carrera que habrían sido muy difíciles de reproducir y depurar, y lo que no estaba previsto en el diseño inicial es que esa misma filosofía, la de empujar la lógica al nivel más bajo posible, también simplificase el módulo de pagos, ya que calcular el IGIC en una Edge Function y no en el cliente hizo que la arquitectura fuera más segura casi sin coste adicional. Si volviera a empezar, habría diseñado desde el principio la máquina de estados de reservas como un trigger y no como validación en el servicio, en lugar de añadirlo en el sprint 8 como corrección.

El análisis de ROI del capítulo 4 demuestra que el modelo es viable bajo hipótesis conservadoras, tomando como referencia 990 plazas activas en el mes 20, 15 reservas por plaza al mes y gastos operativos de 1.130 € al mes una vez superada la fase de captación, lo que permite recuperar los 287.000 € de inversión en el mes 19 con un ROI del 54,7 % a los dos años. El riesgo principal no es técnico ni financiero sino de tracción inicial, dado que el modelo requiere masa crítica de oferta antes de que la demanda se active.

6.1. Líneas de trabajo futuro

Las funcionalidades con mayor impacto sobre la propuesta de valor están descritas en el capítulo 4.2, y la integración real de Stripe Connect junto con la publicación en Google Play y App Store son los dos requisitos no técnicos que bloquean el paso al producto comercial. La línea de crecimiento con mayor potencial es la expansión al resto del archipiélago, dado que el producto no necesita reconstruirse porque la infraestructura técnica, el modelo de negocio y el régimen fiscal del IGIC son idénticos en Las Palmas de Gran Canaria, Arrecife o Santa Cruz de La Palma, lo que hace directamente transferible la experiencia acumulada en el área metropolitana de Tenerife.

Capítulo 7

Summary and Conclusions

Spain has more than 34.7 million registered vehicles and one of the highest vehicle densities in the European Union, and the problem this creates in urban areas is less about an absolute shortage of parking spaces than about a mismatch in their distribution, since public streets are saturated while private garages built under residential and commercial buildings sit empty for most of the working day. In the Canary Islands the situation is compounded by insularity, which limits viable public transport alternatives and makes the private car the dominant mode of urban mobility. Tenerife registers 870 vehicles per thousand inhabitants, the highest rate in the archipelago, and both Santa Cruz de Tenerife and San Cristóbal de La Laguna concentrate most of their commercial, university and administrative activity in historic districts whose street layout predates mass car ownership, leaving very limited room for off-street public parking supply. The ZBE ordinance approved in April 2026, which restricts access to vehicles without a DGT environmental label, adds further pressure by pushing demand toward private spaces in the urban perimeter, a segment that no existing platform is currently structured to serve.

This work aimed to demonstrate, on one hand, that the complete transaction cycle of a peer-to-peer parking marketplace can be resolved within a single application without requiring any coordination between parties outside it, covering space discovery, booking with database-level conflict validation, payment with automatic IGIC calculation and physical garage access, and on the other that doing so produces a commercially viable product within a realistic investment horizon. A subsidiary question was whether the fiscal and regulatory specificity of the Canarian market, in particular the IGIC regime and the ZBE compliance logic, could be handled natively in the architecture rather than worked around.

The system was designed so that the invariants that matter most for correctness are enforced at the lowest possible layer. Booking overlap validation runs as a PL/pgSQL trigger within the database transaction, so conflicting reservations are rejected atomically regardless of the number of concurrent requests, rather than relying on application-level checks that could be bypassed under load. Row Level Security policies in PostgreSQL enforce tenant isolation at the database engine level, independently of the service layer, so a bug in application code cannot expose one user's data to another, while the IGIC calculation and the Stripe Connect payment split are computed in Supabase Edge Functions on the server side, which prevents client-side manipulation of the fiscal breakdown and keeps business logic out of the frontend. Physical garage access is handled through SmartAccess, which generates time-bounded tokens linked to the active booking state and can verify them without a persistent network connection, and the entire product, web application and Android APK alike, is built from a single TypeScript codebase via Capacitor, with React 18 concurrent mode keeping the Leaflet map responsive when rendering large numbers of markers simultaneously.

The commercial study puts the total investment required to bring the prototype to a market-ready product at 287,000 €, covering a professional team across analysis, design, development, testing and deployment. Revenue comes from an asymmetric transaction commission on each booking, with 15 % charged to the driver and 8 % to the owner, which yields a net platform income of approximately 1.77 € per booking after Stripe fees, together with a Parky Pro subscription for multi-space landlords

at a weighted average of 20.99 € per month and premium visibility micropayments averaging 3.19 € per listed space. Under a conservative growth scenario of 100 active spaces at launch growing to approximately 990 by month twenty, with demand stable at 15 bookings per space per month, the model reaches operational break-even in month three and recovers the full initial investment in month nineteen, with an accumulated surplus of 157,038 € at the end of month twenty-four, corresponding to a two-year ROI of 54.7 %.

The results of this work show that a full-featured peer-to-peer parking marketplace is technically feasible within a realistic budget, given that all seven specific objectives defined in Chapter 1 are met by the deployed prototype and that the architectural choices that push correctness invariants to the database layer reduce application complexity rather than just moving it elsewhere. The Canarian fiscal and regulatory specificity that might deter external competitors, most notably the native IGIC handling and the ZBE compliance logic, turns out to be an implementable advantage that raises a meaningful barrier for any platform entering from the mainland. Because the same infrastructure, tax structure and regulatory framework apply across the entire archipelago, geographic expansion does not require rebuilding the product, only replicating the market entry process in cities where the parking density problem is comparable.

7.1. Future work

The most direct growth path is expansion across the Canary Islands archipelago, which requires no architectural changes since the technical infrastructure, business model and regulatory framework are identical in Las Palmas de Gran Canaria, Arrecife and every other island capital. Within the product itself, the extensions with the greatest impact on both revenue and user retention are those described in Chapter 4.2, including passive Bluetooth-based occupancy detection to automate availability updates, event passes and recurring subscriptions via Stripe Billing, automatic ZBE exemption validation, real-time demand-based pricing and a bidirectional reputation system that would address the information asymmetry inherent to any two-sided marketplace in its early stages.

Capítulo 8

Presupuesto

Este capítulo documenta el coste real del desarrollo de este trabajo, diferenciado del estudio de viabilidad del capítulo 4, que modela el escenario comercial con un equipo profesional. Aquí se contabiliza únicamente el esfuerzo invertido por el autor en las distintas fases del proyecto, junto con los equipos utilizados y los servicios necesarios para el despliegue.

8.1. Recursos humanos

El proyecto fue desarrollado íntegramente por un único desarrollador, por lo que el coste de personal se calcula aplicando una tarifa hora uniforme de 18 €/h, correspondiente al coste aproximado de un programador recién graduado en el mercado canario. La tabla 8.1 desglosa las horas dedicadas a cada fase del ciclo de desarrollo, incluyendo la redacción de la memoria, que en un trabajo académico supone una fracción significativa del tiempo total.

Tabla 8.1: Desglose de horas y coste por fase.

Fase	Horas	Coste (€)
Estudio del dominio y análisis de requisitos	30	540,00
Diseño del sistema (arquitectura, BD, UX/UI)	55	990,00
Implementación del backend (BD, RLS, triggers, Edge Functions)	80	1.440,00
Implementación del frontend (React, mapa, flujos de usuario)	90	1.620,00
Integración (Stripe Connect, SmartAccess, APK Android)	60	1.080,00
Pruebas y depuración	45	810,00
Redacción de la memoria	70	1.260,00
Total	440	7.740,00

La fase con mayor carga de trabajo es la redacción de la memoria, equiparable en horas a la implementación del frontend, dado que documentar con precisión las decisiones de arquitectura y el modelo económico requiere un esfuerzo comparable al de su diseño. La implementación del backend concentra la lógica más sensible del proyecto, incluyendo los triggers de solapamiento y las políticas RLS, lo que explica su peso específico a pesar de representar menos líneas de código que el frontend.

8.2. Equipos y material

Los únicos activos materiales necesarios para el desarrollo fueron el sobremesa de trabajo y un dispositivo Android para validar el APK generado por Capacitor. Dado que ambos equipos tienen una vida útil que supera con creces la duración del proyecto, no se contabiliza su coste en este presupuesto.

8.3. Herramientas de software

Todas las herramientas utilizadas durante el desarrollo están disponibles en sus niveles gratuitos o son software libre, por lo que no generan coste directo imputable al proyecto. Visual Studio Code, Git y Android Studio son de descarga libre. Supabase ofrece un plan gratuito con capacidad suficiente para el volumen de un prototipo. Vercel publica la aplicación web sin coste en su plan hobby. Figma permite el diseño de interfaces sin suscripción para proyectos individuales y el plan Free de GitHub cubre tanto el repositorio privado como el historial de versiones completo. El único servicio con coste potencial en producción es el dominio costando 15 euros anuales y Stripe, cuya tarifa del 1,5 % sobre el volumen procesado se aplica únicamente a transacciones reales y, por tanto, no tiene impacto durante la fase de desarrollo con modo sandbox.

8.4. Coste total

Tabla 8.2: Resumen del coste total del proyecto.

Concepto	Coste (€)
Recursos humanos	7.740,00
Equipos y material (amortizado)	0,00
Software y servicios	15,00
Total	7.755,00

El coste total del desarrollo del prototipo asciende a 7.755 €, de los cuales el 97,2 % corresponde al tiempo de trabajo del autor. Esta proporción es coherente con la naturaleza del proyecto, cuya complejidad recae en la configuración de la plataforma y en las decisiones de arquitectura más que en la adquisición de infraestructura, y la elección de herramientas con planes gratuitos permitió mantener el coste de servicios en cero durante todo el ciclo de desarrollo. Este valor contrasta con el presupuesto de 287.000 € proyectado en el capítulo 4 para llevar el mismo producto a escala comercial, diferencia que refleja el paso de un único desarrollador en contexto académico a un equipo multidisciplinar con ciclo de vida profesional completo.

Bibliografía

Las siguientes referencias bibliográficas se presentan en orden de aparición en el texto.

- [1] El País Motor. *Un 35 % del tráfico lo generan conductores que buscan aparcamiento*. 2025. URL: <https://motor.elpais.com/actualidad/un-35-del-trafico-lo-generan-conductores-que-buscan-aparcamiento/>.
- [2] TomTom. *TomTom Traffic Index 2025: Santa Cruz de Tenerife*. TomTom. 2026. URL: <https://www.tomtom.com/traffic-index/city/santa-cruz-de-tenerife>.
- [3] Grand View Research. *Shared Mobility Market Size, Share & Trends Analysis Report*. Grand View Research. 2022. URL: <https://www.grandviewresearch.com/industry-analysis/shared-mobility-market>.
- [4] INRIX Inc. *INRIX 2022 Global Traffic Scorecard*. INRIX Inc. 2022. URL: <https://inrix.com/scorecard/>.
- [5] EasyPark Group. *EasyPark – Ciudades y operadores*. EasyPark Group. 2026. URL: <https://www.easypark.com/es-es/ciudades-y-operadores>.
- [6] Fernando H. Valls. *EasyPark, la app que guía tu aparcamiento*. 4 de dic. de 2024. URL: <https://www.lavanguardia.com/dinero/20241204/10158435/easypark-aparcar-estacionamiento-calle-app.html>.
- [7] Empark Group. *About Us – Empark*. Empark Group. 2026. URL: <https://www.empark.com/es/about-us/>.
- [8] Empark Group. *Telpark – Ciudades con cobertura*. Empark Group. 2026. URL: <https://www.telpark.com/es/ciudades/>.
- [9] Parclick. *Parkings Centro Ciudad, Aeropuerto, Estación y Playa*. Parclick. 2026. URL: <https://parclick.es/>.
- [10] Grupo Mutua Madrileña. *ElParking – Parking App*. Grupo Mutua Madrileña. 2026. URL: <https://www.mutua.es/descuentos/coche/otros/detalle/parking-app/>.
- [11] JustPark Parking Ltd. *About JustPark*. JustPark Parking Ltd. 2026. URL: <https://www.justpark.com/about/>.
- [12] JustPark. *ParkHub Rebrands as JustPark*. JustPark. 2026. URL: <https://www.justpark.com/business/blog/parkhub-rebrands-as-justpark>.
- [13] JustPark Parking Ltd. *How Much Can You Really Earn Renting Out Your Driveway?* JustPark Parking Ltd. 2026. URL: <https://www.justpark.com/resource-centre/how-much-can-you-really-earn-renting-out-your-driveway>.
- [14] SpotHero Inc. *SpotHero: Park Smarter*. SpotHero Inc. 2026. URL: <https://spothero.com/about>.
- [15] Uber Technologies, Inc. *Uber to Acquire Parking App SpotHero*. Uber Technologies, Inc. 2026. URL: <https://investor.uber.com/news-events/news/press-release-details/2026/Uber-to-Acquire-Parking-App-SpotHero/default.aspx>.

- [16] ReservPark Team. *ReservPark – Nunca más pierdas 30 minutos buscando aparcamiento*. ReservPark. 2026. URL: <https://reservpark.com/>.
- [17] Javier Aparicio León. *SocialPark – Compra y vende plazas de aparcamiento en tiempo real*. 2026. URL: <https://socialpark.es/>.
- [18] El País Motor. *El parque móvil crece sin parar desde hace casi cuatro años: cuántos coches hay en España*. 2025. URL: <https://motor.elpais.com/actualidad/el-parque-movil-crece-sin-parar-desde-hace-casi-cuatro-anos-cuantos-coches-hay-en-espana/>.
- [19] *Parque de vehículos. Series históricas*. Dirección General de Tráfico. 2025. URL: <https://www.dgt.es/menusecundario/dgt-en-cifras/dgt-en-cifras-resultados/dgt-en-cifras-detalle/Parque-de-vehiculos-Series-historicas-2025/>.
- [20] Parkiduo. *Ciudades con más atascos en España: cuánto tiempo y dinero pierdes buscando aparcamiento*. 2025. URL: <https://parkiduo.com/2025/04/ciudades-mas-atascos-espana-cuanto-tiempo-dinero-pierdes-buscando-aparcamiento/>.
- [21] Diario de Avisos. *Esto es lo que te costará aparcar en Santa Cruz de Tenerife en 2026*. 2025. URL: <https://diariodeavisos.elespanol.com/2025/01/esto-es-lo-que-te-costara-aparcar-en-santa-cruz-de-tenerife-en-2026/>.
- [22] COPE Tráfico. *El parking de coches más caro de España: llega a costar cinco euros la hora en Barcelona y en torno a 30 si lo dejas 24 horas*. 2026. URL: https://www.cope.es/trafico/audios/parking-coches-caro-espana-llega-costar-cinco-euros-hora-barcelona-torno-30-dejamos-24-horas-20260412_3343633.html.
- [23] Instituto Canario de Estadística (ISTAC). *Estadística de Parque de Vehículos. Series anuales. Islas o municipios de Canarias. 1961–2024*. Gobierno de Canarias. 2025. URL: <https://www.gobiernodecanarias.org/istac/galerias/documentos/E70044A/estadistica-parque-vehiculos-series-mensuales-municipios.html>.
- [24] El Día. *El 35 % de los tinerfenos llega a pie*. 2025. URL: <https://www.eldia.es/tenerife/2025/08/16/35-tinerfenos-llega-pie-15-120664577.html>.
- [25] Jefatura del Estado. *Ley 7/2021, de 20 de mayo, de cambio climático y transición energética*. 2021. URL: <https://www.boe.es/eli/es/l/2021/05/20/7>.
- [26] Ayuntamiento de Santa Cruz de Tenerife. *Zona de Bajas Emisiones (ZBE) de Santa Cruz de Tenerife*. Ayuntamiento de Santa Cruz de Tenerife. 2026. URL: <https://www.santacruzdetenerife.es/web/servicios-municipales/movilidad-y-accesibilidad-universal/zonas-de-bajas-emisiones>.
- [27] idealista. *Alquiler garajes en Santa Cruz de Tenerife*. idealista. 2026. URL: <https://www.idealista.com/alquiler-garajes/santa-cruz-de-tenerife-santa-cruz-de-tenerife/>.
- [28] Microsoft Corporation. *TypeScript: JavaScript with syntax for types*. Microsoft Corporation. 2024. URL: <https://www.typescriptlang.org>.
- [29] Meta Platforms, Inc. *React – The library for web and native user interfaces*. Meta Platforms, Inc. 2024. URL: <https://react.dev>.
- [30] Evan You y Vite Contributors. *Vite – Next Generation Frontend Tooling*. Vite Contributors. 2024. URL: <https://vitejs.dev>.
- [31] Remix Software, Inc. *React Router – Declarative routing for React*. Remix Software, Inc. 2024. URL: <https://reactrouter.com>.

- [32] TanStack. *TanStack Query – Powerful asynchronous state management for TypeScript/JavaScript*. TanStack. 2024. URL: <https://tanstack.com/query>.
- [33] Adam Wathan y Tailwind Labs, Inc. *Tailwind CSS – A utility-first CSS framework*. Tailwind Labs, Inc. 2024. URL: <https://tailwindcss.com>.
- [34] Vladimir Agafonkin. *Leaflet – an open-source JavaScript library for mobile-friendly interactive maps*. 2024. URL: <https://leafletjs.com>.
- [35] OpenCage GmbH. *OpenCage Geocoding API*. OpenCage GmbH. 2024. URL: <https://opencagedata.com>.
- [36] Supabase, Inc. *Supabase – The Open Source Firebase Alternative*. Supabase, Inc. 2024. URL: <https://supabase.com>.
- [37] MongoDB, Inc. *MongoDB: The Developer Data Platform*. MongoDB, Inc. 2024. URL: <https://www.mongodb.com/>.
- [38] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. The PostgreSQL Global Development Group. 2024. URL: <https://www.postgresql.org/>.
- [39] Microsoft. *Visual Studio Code*. 2026. URL: <https://code.visualstudio.com/>.
- [40] Software Freedom Conservancy. *Git*. Software Freedom Conservancy. 2024. URL: <https://git-scm.com/>.
- [41] GitHub, Inc. *GitHub: Let's build from here*. GitHub, Inc. 2024. URL: <https://github.com/>.
- [42] Figma, Inc. *Figma: The collaborative interface design tool*. Figma, Inc. 2024. URL: <https://www.figma.com/>.
- [43] Vercel, Inc. *Vercel – Build and deploy the best web experiences with the frontend cloud*. Vercel, Inc. 2024. URL: <https://vercel.com>.
- [44] One.com. *One.com: Alojamiento web y dominios*. One.com. 2024. URL: <https://www.one.com/>.
- [45] Ionic, Inc. *Capacitor – Cross-platform native runtime for web apps*. Ionic, Inc. 2024. URL: <https://capacitorjs.com>.
- [46] Google LLC. *Firebase*. Google LLC. 2024. URL: <https://firebase.google.com/>.
- [47] Amazon Web Services, Inc. *AWS Amplify*. Amazon Web Services, Inc. 2024. URL: <https://aws.amazon.com/amplify/>.
- [48] Ken Schwaber y Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. 2020. URL: <https://scrumguides.org/scrum-guide.html>.
- [49] Ayuntamiento de Santa Cruz de Tenerife. *Varios 'parking' de Santa Cruz aplicarán tarifas especiales durante Plenilunio*. 2026. URL: <https://www.santacruzdetenerife.es/web/noticias-y-agenda/noticias/detalle-noticia/varios-parking-de-santa-cruz-aplicaran-tarifas-especiales-durante-plenilunio>.
- [50] itm2. *Justpark now charging a fee for withdrawing earnings*. 2026. URL: <https://forums.moneysavingexpert.com/discussion/6569775/justpark-now-charging-a-fee-for-withdrawing-earnings>.